

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«САНКТ-ПЕТЕРБУРГСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
ПЕТРА ВЕЛИКОГО»
ФИЗИКО-МЕХАНИЧЕСКИЙ ИНСТИТУТ
ВЫСШАЯ ШКОЛА ПРИКЛАДНОЙ МАТЕМАТИКИ И ВЫЧИСЛИТЕЛЬНОЙ
ФИЗИКИ

**Отчет о прохождении преддипломной практики
на тему: «Разработка эвристического алгоритма для улучшения сходимости
метода ХРВД в задаче симуляции ткани»**

Парусова Владимира Алексеевича, гр. 5040102/30201

Направление подготовки: 01.04.02 Прикладная математика и информатика.

Место прохождения практики: СПбПУ, ФизМех, ВШПМиВФ.

Руководитель практики от ФГАОУ ВО «СПбПУ»: Беляев Сергей Юрьевич,
доцент ВШПМиВФ, К.Ф.-М.Н.

Консультант практики от ФГАОУ ВО «СПбПУ»: Чуканов Вячеслав Сергеевич,
К.Ф.-М.Н, ВШПМиВФ.

Оценка: Отлично

Руководитель практики
от ФГАОУ ВО «СПбПУ»



С.Ю. Беляев

Консультант практики
от ФГАОУ ВО «СПбПУ»

В.С. Чуканов

Обучающийся

В.А. Парусов

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	3
Глава 1. ПОСТАНОВКА ЗАДАЧИ.....	5
1.1. Техническое задание	5
1.2. Ожидаемый результат	5
Глава 2. ОБЗОР СУЩЕСТВУЮЩИХ РЕШЕНИЙ.....	6
2.1. Алгоритм Position Based Dynamics	6
2.2. Различные улучшения алгоритма Position Based Dynamics	11
2.3. Алгоритм Extended Position Based Dynamics	15
2.4. Задание тканей при помощи ограничений	19
Глава 3. ПРЕДЛАГАЕМОЕ РЕШЕНИЕ	21
3.1. Влияние порядка обхода ограничений на скорость сходимости	21
3.2. Эвристический алгоритм для улучшения сходимости	25
3.3. Оптимизация параллельного обхода ограничений для GPU	32
3.4. Алгоритм нахождения самопересечений	37
3.5. Визуальное представление.....	39
Глава 4. РЕЗУЛЬТАТЫ И ИХ СРАВНИТЕЛЬНЫЙ АНАЛИЗ	42
Заключение	46
Список использованных источников.....	48

ВВЕДЕНИЕ. ОБОСНОВАНИЕ АКТУАЛЬНОСТИ

Компьютерная графика, как часть компьютерных наук, существует уже более семидесяти лет и на данный момент содержит в себе множество различных алгоритмов, позволяющих получать фотореалистичные изображения в режиме реального времени. Однако, несмотря на долгую историю, большая часть упомянутых алгоритмов применима только для симуляции и отображения объектов, представляемых как наборы абсолютно твердых тел.

Относительно недавно был разработан метод Position Based Dynamics (PBD) [14], позволяющий симулировать взаимодействие мягких тел. Данный метод быстро стал стандартом для симуляции таких объектов как ткани, жидкости, желеобразные тела и т.д. Стоит отметить, что данный метод обширно применяется для представления различных органов в программных системах для подготовки к хирургическим операциям [15]. В дальнейшем метод PBD получил множество улучшений, главным из которых можно назвать метод Extended Position Based Dynamics (XPBD) [9], позволяющий использовать табличные физические величины реального мира для задания параметров симуляции мягких тел.

Однако, на практике, даже у тел симулируемых методом XPBD может наблюдаться неестественное поведение, отличающееся от поведения, наблюдаемого в реальном мире. Зачастую, описанное поведение представляет собой периодичные перемещения с большой амплитудой различных частей симулируемого тела (рис. 0.1), и может достигаться резким и сильным взаимодействием с телом (например резко подул сильный ветер). Для того, чтобы бороться с такими ситуациями, в рамках метода XPBD существует два решения. Первое - поставить условия симуляции таким образом, чтобы подобных взаимодействий не возникало, однако это не всегда возможно. Второе - проводить вычисления с большей точностью, что будет требовать больше вычислительных ресурсов и зачастую неприемлемо в условиях симуляций реального времени.



Рис.0.1. Пример неестественного растяжения в результате сильного ветра.
Кадры сняты спустя 1, 2 и 3 секунды после запуска симуляции.

В рамках данной работы, была поставлена цель разработать и реализовать эвристический алгоритм, позволяющий бороться с вышеописанной ситуацией, при этом не требуя много больше процессорного времени. Помимо этого, была поставлена цель реализации метода XPBD с использованием GPU, и последующее сравнение предлагаемых алгоритмов с реализацией из библиотеки NVidia Cloth.

ГЛАВА 1. ПОСТАНОВКА ЗАДАЧИ

1.1. Техническое задание

Требуется разработать и реализовать алгоритмы и их модификации, применимые в современных графических конвейерах, решающие в реальном времени следующие задачи:

- симуляция тканей методом XPBD;
- улучшение сходимости метода XPBD с помощью эвристического алгоритма;
- улучшение производительности метода XPBD с учетом архитектурных особенностей GPU и используемого графического конвейера;
- задача коллизии тканей с плоскостями, сферами, капсулами;
- задача коллизии тканей с другими тканями.

1.2. Ожидаемый результат

Ожидаемым результатом работы является улучшение проекта “DX12Engine” за счет добавления в него функциональности, осуществляющей симуляцию поведения тканей. При этом, в результирующем проекте должна присутствовать возможность переключения в реальном времени алгоритма симуляции между тремя вариантами:

- XPBD использующий GPU;
- XPBD использующий GPU, совмещенный с эвристическим алгоритмом;
- алгоритм симуляции представленный в библиотеке NVidia Cloth.

Помимо этого, ожидаемым результатом является сравнительный анализ реализованных алгоритмов как с точки зрения производительности, так и с точки зрения визуальной корректности.

ГЛАВА 2. ОБЗОР СУЩЕСТВУЮЩИХ РЕШЕНИЙ

2.1. Алгоритм Position Based Dynamics

Предложенный в 2007 году алгоритм Position Based Dynamics[14] является оснополагающим для большинства современных алгоритмов симуляции тканей. Рассмотрим принципы работы и математические модели, лежащие в основе данного метода.

В рамках предложенной математической модели, любое симулируемое тело представляет собой набор из N частиц, соединенных M ограничениями. При этом **состояние частицы i** характеризуется следующими параметрами:

1. Положение частицы (в момент времени t) $p_i(t)$
2. Обратная масса $im_i = 1/m_i$

Стоит отметить две важные особенности рассматриваемой математической модели. Во-первых, размер частицы считается пренебрежимо малым по отношению к размеру всего тела. Во-вторых, для описания состояния частицы не используются скорость и ускорение данной частицы (в отличие от методов симуляции, используемых для обработки физического взаимодействия абсолютно твердых тел).

Под **ограничением j** понимается набор из:

1. Размерности ограничения (количество затрагиваемых частиц) n_j
2. Функции $C_j : \mathbb{R}^{n_j} \rightarrow \mathbb{R}$
3. Набор индексов частиц, к которым применяется данное ограничение $\{i_1, i_2, \dots, i_{n_j}\}, i_k \in [1, \dots, N]$
4. Параметр жесткости $k_j \in [0 \dots 1]$
5. Тип “равенство” или “неравенство”

Будем считать, что ограничение j типа “равенство” выполнено (удовлетворено), когда выполняется следующее условие:

$$C_j(p_{i_1}, p_{i_2}, \dots, p_{i_{n_j}}) = 0 \quad (2.1)$$

Если же ограничение j имеет тип “неравенство”, тогда будем считать, что оно выполнено (удовлетворено), когда выполняется:

$$C_j(p_{i_1}, p_{i_2}, \dots, p_{i_{n_j}}) \geq 0 \quad (2.2)$$

В качестве примера можно привести ограничение “пружина”, использующееся для симуляции ткани. Функция C_j такого ограничения будет выглядеть

следующим образом:

$$C_j(p_{i_1}, p_{i_2}) = |p_{i_1} - p_{i_2}| - d \quad (2.3)$$

В этом выражении переменная d обозначает длину “пружины” в состоянии покоя. В дальнейшей работе будут рассматриваться только ограничения вида “равенство”, однако ограничение типа “неравенство” может быть приведено к ограничению типа “равенство” при помощи построения новой функции ограничения $C'_j = \max(0, C_j)$.

Тогда, если частицы и их ограничения заданы вышеописанным способом, алгоритм PBD будет представляться следующим образом (рис. 2.1).

Algorithm

Input: время шага симуляции Δt , количество итераций $solverIteration$, текущее состояние частиц, предыдущее состояние частиц, ограничения, функция суммы внешних сил от положения $f_{ext}(p)$

Output: положение частиц спустя заданное время $\{p_i(t + \Delta t)\}_N$

1. **for** $i \in 1..N$ **do**
2. $v_i = \frac{p_i(t) - p_i(t - \Delta t)}{\Delta t} + \Delta t * im_i * f_{ext}(p_i);$
3. **for** $i \in 1..N$ **do**
4. $p_i^* = p_i(t) + v_i * \Delta t;$
5. **for** $k \in 1..solverIteration$ **do**
6. $p_1^*, ..., p_N^* = projectConstraints(p_1^*, ..., p_N^*);$
7. **for** $i \in 1..N$ **do**
8. $p_i(t + \Delta t) = p_i^*;$

Рис.2.1. Псевдокод алгоритма Position Based Dynamics

Как можно заметить, данный алгоритм является итеративным и определяет новое состояние тела из текущего состояния и величины прошедшего времени. Начинается алгоритм с предсказания новых положений частиц путем численного интегрирования уравнения перемещения частицы методом Стёрмера-Верле [19] (строка 3). Данный метод был выбран потому, что для него достаточно только одного предыдущего значения, а также он является более устойчивым, чем метод Эйлера. Далее (строка 5) несколько раз (указанное в аргументе $solverIteration$) запускается алгоритм $projectConstraints$, который модифицирует предсказан-

ные положения частиц таким образом, чтобы приблизиться к выполнению всех поставленных ограничений. Затем (строка 7) положения частиц на момент времени $t + \Delta t$ принимаются равными предсказанным позициям.

Для того, чтобы описать принцип работы алгоритма *projectConstraints*, необходимо найти способ решать такую задачу для одного ограничения. Для этого возьмем произвольное ограничение и обозначим функцию этого ограничения C . Поставим задачу найти вектор Δp такой, что:

$$C(p + \Delta p) = 0 \quad (2.4)$$

Для этого достаточно разложить данное равенство в ряд Тейлора:

$$C(p + \Delta p) \approx C(p) + \Delta p * \nabla_p C(p) = 0 \quad (2.5)$$

Преобразовав полученное равенство и выделив множитель *scaleFactor*, общий для всех связанных данным ограничением частиц, получим:

$$scaleFactor = -\frac{C(p)}{|\nabla C(p)|^2} \quad (2.6)$$

$$\Delta p_i = scaleFactor * \nabla_{p_i} C(p) \quad (2.7)$$

Таким образом, используя выражение 2.7, мы для каждой связанной ограничением частицы можем найти смещение, необходимое для удовлетворения данного ограничения, но только в случае, если массы частиц равны между собой. Иначе требуется воспользоваться приведенным ниже выражением 2.8:

$$scaleFactor = -\frac{1}{\sum_l im_l} \frac{C(p)}{|\nabla C(p)|^2} \quad (2.8)$$

$$\Delta p_i = im_i * scaleFactor * \nabla_{p_i} C(p) \quad (2.9)$$

Стоит отметить, что при значении $im_i = 0$ (соответствует бесконечной массе) не только вектор смещения $\Delta p_i = 0$, но и внешние силы при интеграции методом Стёрмера-Верле (рис. 2.1) не будут влиять на положение вершины. Это означает, что таким образом можно “закреплять” такие вершины в пространстве, или “прикреплять” к другим симулируемым физическим телам.

Algorithm *solveConstraint*

- Input:** размерность ограничения n , функция ограничения C , текущие положения частиц $\{p_i\}_n$, обратные массы частиц $\{im_i\}_n$
- Output:** вектора смещения частиц $\{\Delta p_i\}_n$ для удовлетворения заданному ограничению
1. $scaleFactor = C(p_1, \dots) / \sum_i (im_i * |\nabla_{p_i} C(p_1, \dots, p_n)|^2);$
 2. **for** $i \in 1..n$ **do** $\Delta p_i = -scaleFactor * im_i * \nabla_{p_i} C(p_1, \dots, p_n);$

Рис.2.2. Псевдокод алгоритма *solveConstraint*

Таким образом получаем алгоритм *solveConstraint* (рис.2.2) для решения задачи в случае одного ограничения. Для того, чтобы решить задачу для набора ограничений, авторами было предложено 2 алгоритма.

Первый (рис.2.3), по словам авторов, основан на методе Якоби для решения СЛАУ, хотя функции ограничений не всегда являются линейными. Идея данного алгоритма заключается в том, чтобы для каждого ограничения вычислить смещения соответствующих вершин и сохранить их во временную переменную. Затем для каждой частицы прибавить к её положению значение суммарного смещения, сохранённое во временной переменной, умноженный на некоторую скалярную константу γ .

Algorithm *projectConstraintsJacobi*

- Input:** константа γ , ограничения, текущее состояние частиц
- Output:** новые положения частиц $\{p_i^*\}_N$
1. **for** $i \in 1..N$ **do** $o_i = 0;$
 2. **for** $j \in 1..M$ **do**
 3. $tmp = solveConstraint(n_j, C_j, p_{i_1}, \dots, p_{i_{n_j}}, im_{i_1}, \dots, im_{i_{n_j}});$
 4. **for** $l \in 1..n_j$ **do** $o_{i_l} = o_{i_l} + k_j * tmp_l;$
 5. **for** $i \in 1..N$ **do** $p_i^* = p_i + \gamma * o_i;$

Рис.2.3. Псевдокод алгоритма *projectConstraints* использующего метод Якоби

Второй (рис.2.4), по словам авторов, основан на методе Гаусса-Зейделя для решения СЛАУ. Данный метод отличается от предыдущего тем, что смещения применяются к частицам сразу, а не накапливаются.

Algorithm *projectConstraintsGauss***Input:** ограничения, текущее состояние частиц**Output:** новые положения частиц $\{p_i^*\}_N$

1. **for** $j \in 1..M$ **do**
2. $tmp = solveConstraint(n_j, C_j, p_{i_1}, \dots, p_{i_{n_j}}, im_{i_1}, \dots, im_{i_{n_j}});$
3. **for** $l \in 1..n_j$ **do** $p_i^* = p_i + k_j * tmp_l;$

Рис.2.4. Псевдокод алгоритма *projectConstraints* использующего метод Гаусса-Зейделя

Как можно заметить, оба алгоритма ориентируются на итеративные алгоритмы решения СЛАУ. Для выбора между этими двумя алгоритмами, авторы оригинальной статьи поставили мысленный эксперимент (рис.2.5):

1. Рассматриваются три частицы p_1, p_2, p_3 . Частицы p_1, p_3 закреплены в пространстве.
2. Рассматривается два ограничения “пружина”: $C_1(p_1, p_2)$ с длиной покоя l_1 и $C_2(p_2, p_3)$ с длиной покоя l_2 .
3. Исходя из приведенного на рисунке положения частиц, видно, что оптимальным положением частицы p_2 является точка пересечения пунктирных дуг, имеющих радиусы l_1 и l_2 .
4. Проводится одна итерация алгоритмом, основанным на методе Якоби, и одна итерация алгоритмом, основанным на методе Гаусса-Зейделя
5. В результате поставленного эксперимента можно наблюдать, что первый алгоритм сходится медленнее, чем второй.

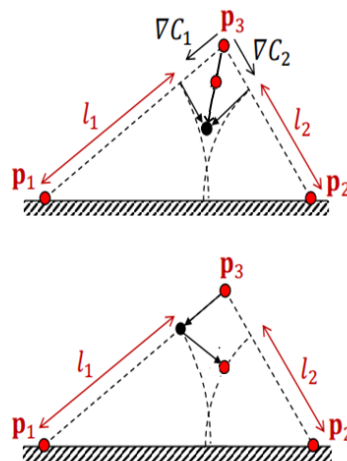


Рис.2.5. Эксперимент поставленный авторами статьи.

На верхнем изображении представлен результат первой итерации алгоритмом основанным на методе Якоби, на нижнем изображении результат первой итерации алгоритмом основанным на методе Гаусса-Зейделя

Помимо мысленных экспериментов, авторами были поставлены и численные эксперименты с использованием компьютера. Согласно полученным ими результатам, первый алгоритм действительно сходится медленнее, для его реализации требуется выделение дополнительной памяти и подбор значения константы γ . В качестве преимуществ авторы отмечают возможность и простоту параллельной реализации. С другой стороны, алгоритм, основанный на методе Гаусса-Зейделя, сходится быстрее, его реализация не требует дополнительного объема памяти, но полученный результат будет зависеть от порядка обхода ограничений. Возможность параллельной реализации данного алгоритма зависит от выбора порядка обхода ограничений.

2.2. Различные улучшения алгоритма Position Based Dynamics

Алгоритм, описанный в предыдущей главе, позволяет симулировать поведение мягких тел, однако не может быть применен на практике из-за отсутствия обработки взаимодействия с другими симулируемыми телами, а также большим временным затратам. В связи с этим, в статье [14], помимо основной идеи алгоритма, были предложены различные улучшения.

Во-первых, требуется исправить то, что симулируемое описанным образом мягкое тело не взаимодействует с другими физическими телами. Это означает, что при визуализации симулируемого тела можно будет наблюдать, “прохождение сквозь” другие тела. Для недопущения таких ситуаций, используются алгоритмы обнаружения и обработки столкновений. В статье [14] для решения этой проблемы предлагается динамически создавать новые ограничения с параметрами:

1. Функция $C(p) = (p - q) * n$, где q - точка пересечения луча, выходящего из положения частицы p в направлении предсказанного положения частицы p^* , с поверхностью твердого объекта, а n - вектор нормали к поверхности данного объекта в точке q .
2. Параметр жесткости $k = 1$.
3. Тип ограничения “неравенство”.

Таким образом получается достичь “одностороннего взаимодействия” - мягкое тело отталкивается от твердых, но твердые тела не изменяют свои импульсы в результате столкновений с мягкими. Чтобы добиться “двухстороннего взаимодействия”, авторами было принято решение придавать твёрдому телу импульс, равный $\frac{m_i * (p_i(t + \Delta t) - p_i(t))}{\Delta t}$ за каждую частицу, для которой было обнаружено пересечение.

Во-вторых, необходимо исправить такую проблему: симулируемое мягкое тело не взаимодействует с частями самого себя. То есть если частицы не соединены ограничениями напрямую, то эти частицы могут свободно перемещаться друг относительно друга, что визуально приводит к самопересечениям. В статье [14] авторы предлагают разбить поверхность мягкого тела на треугольники и проверять пересечение частиц, с треугольниками заданными тремя вершинами p_1^t, p_2^t, p_3^t и толщиной h , при помощи функций:

$$C(p^*, p_1^t, p_2^t, p_3^t) = (p^* - p_1^t) \cdot \frac{(p_2^t - p_1^t) \times (p_3^t - p_1^t)}{|(p_2^t - p_1^t) \times (p_3^t - p_1^t)|} - h \quad (2.10)$$

Или, если частица входит в треугольник со стороны обратной стороне нормали:

$$C(p^*, p_1^t, p_2^t, p_3^t) = (p^* - p_1^t) \cdot \frac{(p_3^t - p_1^t) \times (p_2^t - p_1^t)}{|(p_3^t - p_1^t) \times (p_2^t - p_1^t)|} - h \quad (2.11)$$

Таким образом возможно избавиться от самопересечений, однако, если просто проверять все возможные частицы со всеми возможными треугольниками, данный алгоритм окажется излишне нагруженным с точки зрения времени выполнения. Для того, чтобы убрать из рассмотрения часть проверяемых треугольников, авторами используется алгоритм пространственного хеширования треугольников, описанный в [13].

В-третьих, для удаления излишних колебаний, авторы используют демпфирование, уменьшая скорости, полученные на этапе предсказания положений частиц, используя константу демпфирования.

Применив вышеописанные улучшения, достигается возможность симулировать поверхность ткани, разрываемой в результате взаимодействия с физическими объектами, как показано на рис. 2.6. Результирующий алгоритм представлен на рис. 2.7.

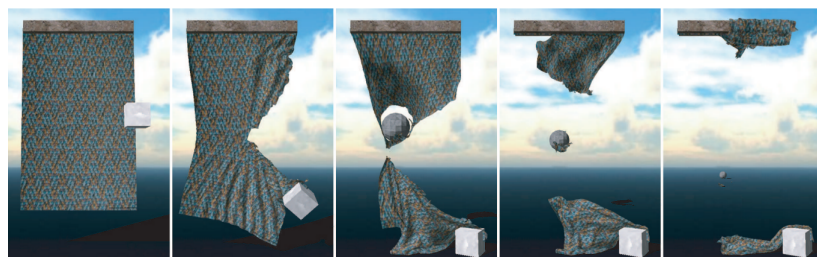


Рис.2.6. Пример куска ткани, разорванного в следствии взаимодействия с летящей сферой и закрепленным кубом.

Algorithm

Input: время шага симуляции Δt , количество итераций $solverIteration$, текущее и предыдущее состояние частиц, ограничения, функция внешних сил от положения $f_{ext}(p)$,

Output: положение частиц спустя заданное время $\{p_i(t + \Delta t)\}_N$

1. **for** $i \in 1..N$ **do** $v_i = \frac{p_i(t) - p_i(t - \Delta t)}{\Delta t} + \Delta t * im_i * f_{ext}(p_i)$;
2. $dampVelocities(v_1, \dots, v_N)$;
3. **for** $i \in 1..N$ **do** $p_i^* = p_i(t) + v_i * \Delta t$;
4. **for** $i \in 1..N$ **do** $genCollisionConstraints(p_i - > p_i^*)$;
5. **for** $k \in 1..solverIteration$ **do**
6. $p_1^*, \dots, p_N^* = projectConstraints(p_1^*, \dots, p_N^*)$;
7. **for** $i \in 1..N$ **do** $p_i(t + \Delta t) = p_i^*$;

Рис.2.7. Псевдокод алгоритма Position Based Dynamics с учетом коллизий и демпфирования

Как можно заметить, временная сложность данного алгоритма пропорциональна произведению числа частиц на число итераций. В связи с этим, авторы многих статей ставили перед собой цель уменьшить два этих параметра, при этом сохраняя точность.

Так, в статье [12] авторами было замечено, что при увеличении размерности сетки, основным визуальным улучшением являются небольшие складки, не сильно влияющие на глобальное поведение ткани. Тогда было решено разделить ткань на глобальную сетку с низким числом частиц и локальную сетку с высоким числом частиц. Основной алгоритм симуляции ткани (вместе с обработкой пересечений) применялся к глобальной сетке. Локальная сетка же, прикреплялась к глобальной сетке, и её симуляция не требовала большого количества итераций (рис.2.8 и рис.2.9).

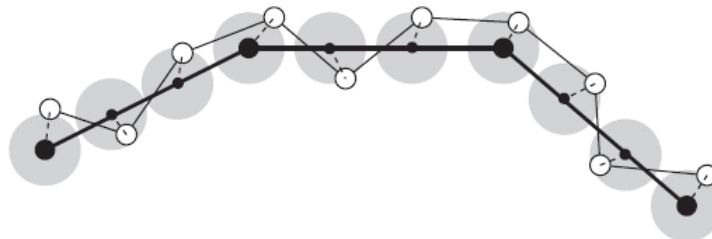


Рис.2.8. Схема, поясняющая идею алгоритма складок. Черные точки - вершины глобальной сетки, белые точки - вершины локальной сетки. Прикрепление локальной сетки к глобальной происходит за счет добавления ограничений на расстояние, представленных в виде серых дисков

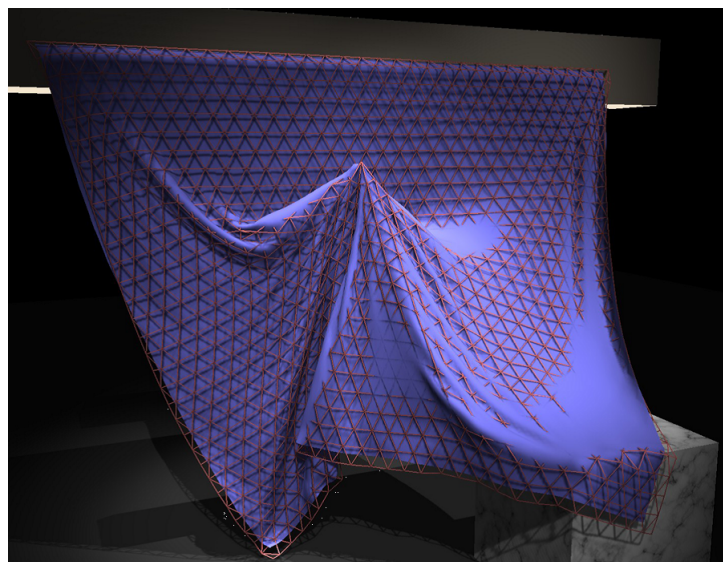


Рис.2.9. Визуальная демонстрация алгоритма складок. Красным представлена глобальная сетка, а фиолетовым представлена ткань локальной сетки

А в другой статье [8] авторами была предпринята попытка решить проблему глобального растяжения ткани в случае малого числа итераций. Для этого они разработали технику Long Range Attachments (LRA). Она основывается на том, что зачастую растяжение ткани проявляется в ситуациях, когда какие-либо частицы ткани закреплены в пространстве. Идея данной техники заключается в том, чтобы связывать ограничения не только соседние вершины. Добавив дополнительные ограничения, связывающие закрепленные вершины с другими вершинами (как показано на рис.2.10) можно ограничить смещения незакрепленных частиц относительно закрепленных, тем самым избавляясь от излишнего растяжения.

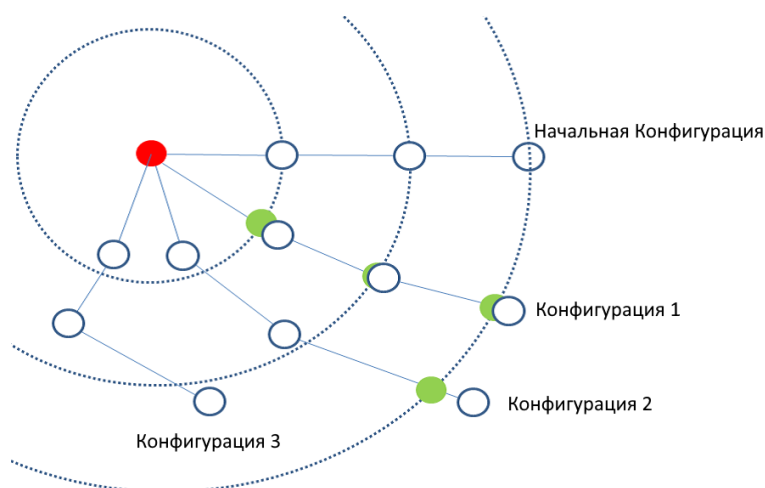


Рис.2.10. Схема, поясняющая идею алгоритма Long Range Attachments на примере нерастяжимой веревки. Красным цветом обозначена закрепленная вершина. Каждая другая вершина удерживается или остается внутри соответствующей ей сферы (представленной пунктиром). Для каждой конфигурации (если вершина оказалась за пределами своей сферы) зеленым представлено положение, исправленное алгоритмом LRA

2.3. Алгоритм Extended Position Based Dynamics

В статье [3] авторами оригинальной статьи Position Based Dynamics были обозначены шесть основных проблем данного алгоритма:

1. Числа, используемые в качестве параметров, не являются физическими величинами.
2. Жесткость симулируемого мягкого тела зависит как от количества итераций, так и от величины временного шага.
3. Интегрирование не является физически точным.
4. Результат зависит от порядка.
5. Результат зависит от геометрической структуры ткани.
6. Алгоритм медленно сходится.

Для решения первых трех проблем авторами был разработан алгоритм Extended Position Based Dynamics [9], являющийся улучшением оригинального. Для этого авторы взяли известное уравнение

$$\mathbf{M}\ddot{\mathbf{x}} = -\nabla U^T(\mathbf{x}) \quad (2.12)$$

В данном уравнении под $\mathbf{M}$ понимаются матрица с массами частиц, под $\mathbf{x}$ вектор из положений частиц, а под U - потенциальная энергия частиц. Если дискретизировать данное уравнение по времени, то мы получим следующее уравнение:

$$\mathbf{M}\left(\frac{\mathbf{x}^{n+1} - 2\mathbf{x}^n + \mathbf{x}^{n-1}}{\Delta t^2}\right) = -\nabla U^T(\mathbf{x}^{n+1}) \quad (2.13)$$

Записав потенциальную энергию через функции ограничения, описанные в [17], мы получаем систему уравнений:

$$\mathbf{M}(\mathbf{x}^{n+1} - 2\mathbf{x}^n + \mathbf{x}^{n-1}) - \nabla \mathbf{C}(\mathbf{x}^{n+1})^T \boldsymbol{\lambda}^{n+1} = 0 \quad (2.14)$$

$$\mathbf{C}(\mathbf{x}^{n+1}) + \frac{\alpha}{\Delta t^2} \boldsymbol{\lambda}^{n+1} = 0 \quad (2.15)$$

В данной системе $\mathbf{C} = [C_1(\mathbf{x}), C_2(\mathbf{x}), \dots, C_M(\mathbf{x})]^T$ - вектор, состоящий из функций ограничения, $\boldsymbol{\lambda} = [\lambda_1, \lambda_2, \dots, \lambda_M]^T$ - вектор множителей ограничений, α - диагональная матрица содержащая значения, обратные величине жесткости каждого ограничения. Если мы обозначим выражение 2.14 как $g(\mathbf{x}, \boldsymbol{\lambda}) = 0$, а выражение 2.15 как $h(\mathbf{x}, \boldsymbol{\lambda}) = 0$, то, воспользовавшись методом Ньютона для решения полученной системы, мы получаем систему вида:

$$\begin{bmatrix} \frac{\delta g}{\delta x} & -\nabla \mathbf{C}^T(x_i) \\ \nabla \mathbf{C}(x_i) & \frac{\alpha}{\Delta t^2} \end{bmatrix} \begin{bmatrix} \Delta x \\ \Delta \lambda \end{bmatrix} = - \begin{bmatrix} g(x_i, \lambda_i) \\ h(x_i, \lambda_i) \end{bmatrix} \quad (2.16)$$

Далее, авторы статьи вносят два допущения. Первое - принимается что $\frac{\delta g}{\delta x} = M$. Это исключает геометрическую жесткость и вносит погрешность порядка $O(\Delta t^2)$ (данный метод тогда можно рассматривать как квази-Ньютоновский). Второе - $g(x_i, \lambda_i) = 0$. Хотя это верно только для первой итерации метода Ньютона (при $x_0 = 2x^n + x^{n-1}$, $\lambda_0 = 0$), если $\nabla \mathbf{C}^T$ будет изменяться медленно, то значение $g(x_i, \lambda_i)$ будет отставаться малым, и станет равно нулю, когда $\nabla \mathbf{C}^T$ станет константным. Если внести эти две аппроксимации, то система 2.17 будет иметь вид:

$$\begin{bmatrix} \mathbf{M} & -\nabla \mathbf{C}^T(x_i) \\ \nabla \mathbf{C}(x_i) & \frac{\alpha}{\Delta t^2} \end{bmatrix} \begin{bmatrix} \Delta x \\ \Delta \lambda \end{bmatrix} = - \begin{bmatrix} 0 \\ h(x_i, \lambda_i) \end{bmatrix} \quad (2.17)$$

Взяв дополнение Шура [22] мы можем получить следующую систему относительно $\Delta \lambda$:

$$\left[\nabla \mathbf{C}(x_i) \mathbf{M}^{-1} \nabla \mathbf{C}^T(x_i) + \frac{\alpha}{\Delta t^2} \right] \Delta \lambda = -\mathbf{C}(x_i) - \frac{\alpha}{\Delta t^2} \lambda_i \quad (2.18)$$

Значение Δx может быть вычислено с помощью следующего выражения:

$$\Delta x = \mathbf{M}^{-1} \nabla \mathbf{C}^T(x_i) \Delta \lambda \quad (2.19)$$

Заметим, что выражение 2.19 совпадает с выражением 2.9 описанным в параграфе 2.1, если принять $\Delta \lambda$ равной *scaleFactor*. Перепишем выражение 2.18 для $\Delta \lambda$ в форме, аналогичной представлению *scaleFactor* в равенстве 2.8.

$$\Delta \lambda = \frac{-\mathbf{C}(x_i) - \frac{\alpha}{\Delta t^2} \lambda_i}{\nabla \mathbf{C}(x_i) \mathbf{M}^{-1} \nabla \mathbf{C}^T(x_i) + \frac{\alpha}{\Delta t^2}} \quad (2.20)$$

Данное выражение отличается от представленного 2.8 тем, что в числителе появилось слагаемое $-\frac{\alpha}{\Delta t^2} \lambda_i$, а в знаменателе $\frac{\alpha}{\Delta t^2}$. Именно эти два слагаемых позволяют отказаться от используемого в алгоритме PBD параметра жесткости k , не сводящегося к реальным физическим величинам, и использовать вместо него физически корректное значение жесткости, определяемое с помощью известных табличных величин реального мира. Заметим, что оба вышеприведенных слагаемых также зависят от промежутка времени, тем самым внося в рассчитанную жесткость поправку в соответствии с выбранным временным шагом. Помимо этого, на каждой итерации алгоритма используется разное значение λ_i , что также нейтрализует

зависимость жесткости от количества итераций. При этом, если рассмотреть ситуацию, при которой параметр жесткости равен бесконечности, то матрица α будет равна нулевой, и алгоритм XPBD сведется к алгоритму PBD.

Общий вид алгоритма XPBD представлен на рис. [2.11](#).

Algorithm

Input: время шага симуляции Δt , количество итераций $solverIteration$, текущее состояние частиц, предыдущее состояние частиц, ограничения, функция суммы внешних сил $f_{ext}(p)$

Output: положение частиц спустя заданное время $\{p_i(t + \Delta t)\}_N$

1. **for** $i \in 1..N$ **do** $v_i = \frac{p_i(t) - p_i(t - \Delta t)}{\Delta t} + \Delta t * im_i * f_{ext}(p_i)$;
2. $dampVelocities(v_1, \dots, v_N)$;
3. **for** $j \in 1..M$ **do** $\lambda_j = 0$;
4. **for** $i \in 1..N$ **do** $p_i^* = p_i(t) + v_i * \Delta t$;
5. **for** $i \in 1..N$ **do** $genCollisionConstraints(p_i - > p_i^*)$;
6. **for** $k \in 1..solverIteration$ **do**
7. $p_1^*, \dots, p_N^* = projectXPBDConstraints(\Delta t, p_1^*, \dots, p_N^*, \lambda_1, \dots, \lambda_M)$;
8. **for** $i \in 1..N$ **do** $p_i(t + \Delta t) = p_i^*$;

Рис.2.11. Псевдокод алгоритма Extended Position Based Dynamics

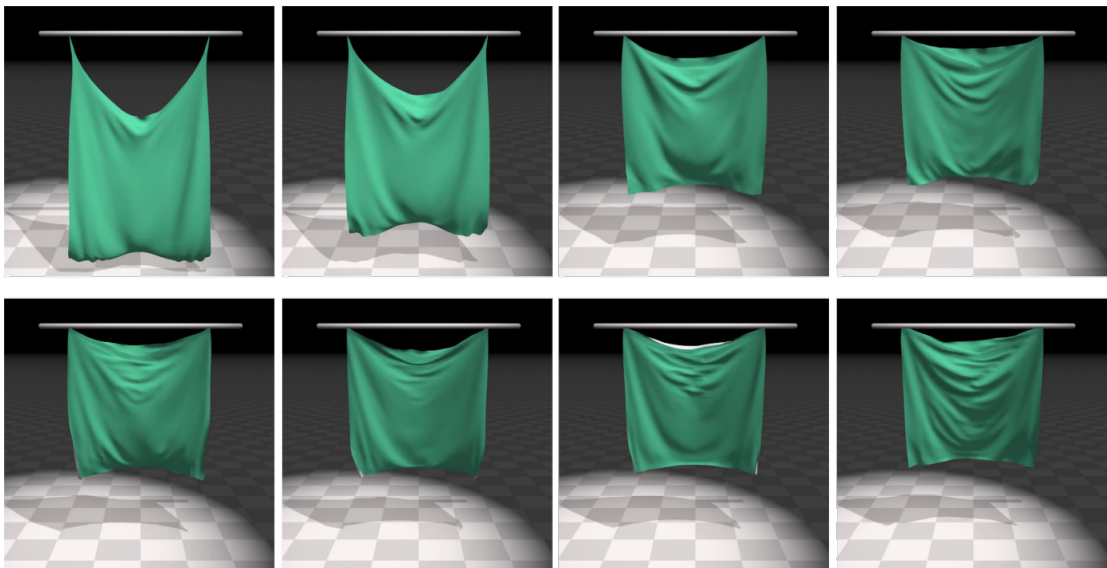


Рис.2.12. Сравнение работы алгоритмов PBD и XPBD на примере свисающей ткани спустя 20, 40, 80, 160 итераций (слева направо). Верхняя строка PBD, нижняя XPBD. Можно заметить, что растяжение ткани в алгоритме PBD нелинейно зависит от числа итераций, тогда как растяжение ткани при использовании алгоритма XPBD качественно неизменна

Соответствующий алгоритм *solveXPBDConstraint* представлен на рис. 2.13

Algorithm *solveXPBDConstraint*

- Input:** время шага симуляции Δt , значения α и λ для ограничения, размерность ограничения n , функция ограничения C , текущие положения частиц $\{p_i\}_n$, обратные массы частиц $\{im_i\}_n$,
- Output:** вектора смещения частиц $\{\Delta p_i\}_n$ для удовлетворения заданному ограничению, новое значение λ
1. $s = \frac{\alpha}{\Delta t^2}$;
 2. $scaleNum = -C(p_1, \dots) - s\lambda$;
 3. $scaleDenum = \sum_i (im_i * |\nabla_{p_i} C(p_1, \dots, p_n)|^2) + s$;
 4. $scaleFactor = scaleNum / scaleDenum$;
 5. $\lambda = \lambda + scaleFactor$;
 6. **for** $i \in 1..n$ **do** $\Delta p_i = scaleFactor * im_i * \nabla_{p_i} C(p_1, \dots, p_n)$; ;

Рис.2.13. Псевдокод алгоритма *solveXPBDConstraint*

Единственным недостатком данного алгоритма является то, что для его исполнения требуется дополнительная память для хранения параметров λ . Однако, вскоре авторами была выпущена ещё одна статья [18], в которой ими было обнаружено, что вместо того, чтобы увеличивать количество итераций *solverIteration*, более эффективно запускать весь алгоритм несколько раз (обозначим это количество как *solverStep*), с уменьшенным промежутком времени. Таким образом, ими была предложена техника Small Steps, представленная на рис. 2.15. При этом, крайний случай, при котором *solverIteration* = 1, не только является корректным и достаточно стабильным, но и позволяет избавиться от дополнительных затрат памяти и процессорного времени на параметры λ , так как на первой итерации они равны 0.

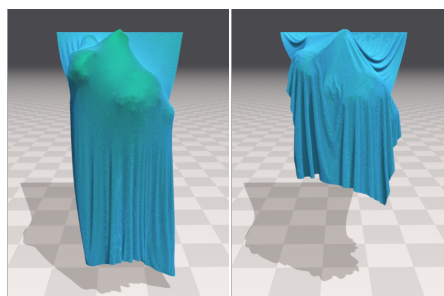


Рис.2.14. Сравнение XPBD с использованием техники Small Steps с различными параметрами.

Слева *solverIteration* = 30, *solverStep* = 1, справа *solverIteration* = 1, *solverStep* = 30

Algorithm

Input: время шага симуляции Δt , количество шагов $solverStep$, количество итераций $solverIteration$, текущее состояние частиц, предыдущее состояние частиц, ограничения, функция суммы внешних сил $f_{ext}(p)$

Output: положение частиц спустя заданное время $\{p_i(t + \Delta t)\}_N$

```

1.  $st = \Delta t / solverStep$ ;
2. for  $step \in 1..solverStep$  do
3.   for  $i \in 1..N$  do  $v_i = \frac{p_i(t+(step-1)*st) - p_i(t-(step-2)*st)}{st} + st * im_i * f_{ext}(p_i)$ ;
4.    $dampVelocities(v_1, \dots, v_N)$ ;
5.   for  $j \in 1..M$  do  $\lambda_j = 0$ ;
6.   for  $i \in 1..N$  do  $p_i^* = p_i(t) + v_i * st$ ; ;
7.   for  $i \in 1..N$  do  $genCollisionConstraints(p_i - > p_i^*)$ ; ;
8.   for  $k \in 1..solverIteration$  do
9.      $p_1^*, \dots, p_N^* =$ 
10.     $projectXPBDConstraints(st, p_1^*, \dots, p_N^*, \lambda_1, \dots, \lambda_M)$ ;
10.   for  $i \in 1..N$  do  $p_i(t + step * st) \leftarrow p_i^*$ ;

```

Рис.2.15. Псевдокод алгоритма Extended Position Based Dynamics с использованием техники Small Steps

2.4. Задание тканей при помощи ограничений

Ещё одна важная тема, которую необходимо осветить - способы задания различных мягких тел при помощи ограничений. Данная работа посвящена симуляции тканей, поэтому будут рассмотрены только способы их задания. В современных системах симуляции тканей используется два основных подхода: с использованием прямоугольной сетки и с использованием триангулированной геометрии.

Для первого способа используются следующие ограничения:

1. Ограничение растяжения/структурное ограничение (Structural/streching). Определяется как ограничение “пружина”, соединяющее соседние частицы как показано на рис.2.16. Задаёт структуру всей ткани.
2. Ограничение сдвига (Shear). Определяется как ограничение “пружина” соединяющее диагональные частицы как показано на рис.2.17. Вводится для корректной симуляции диагональных растяжений (рис.2.19).

3. Ограничение изгиба (Bending). Может определяться как ограничение угла [20] или как ограничение “пружина”, соединяющее частицы “через одну”, как показано на рис. 2.18.

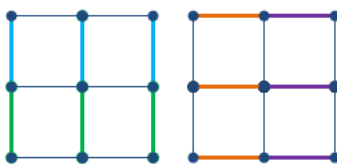


Рис.2.16. Визуальная демонстрация структурных ограничений

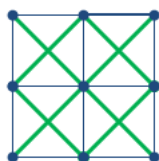


Рис.2.17. Визуальная демонстрация ограничений сдвига

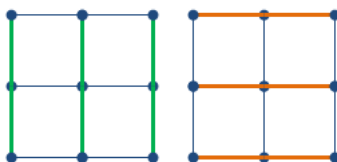


Рис.2.18. Визуальная демонстрация ограничений изгиба

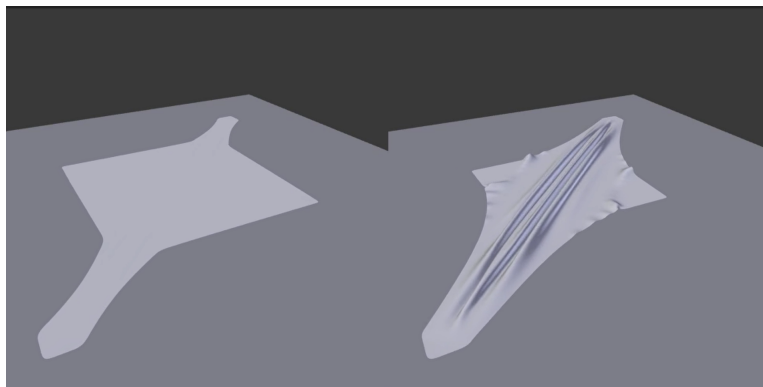


Рис.2.19. Сравнение тканей с отсутствием ограничения сдвига (слева) и наличием ограничения сдвига (справа)

При использовании второго подхода, триангулированная геометрия проходит через предобработку, в которой вышеописанные ограничения (для прямоугольной сетки) накладываются на вершины триангулированной геометрии. Для этого триангулированная геометрия разбивается на прямоугольные сетки, а затем эти прямоугольные сетки “сшиваются” между собой при помощи структурных ограничений.

ГЛАВА 3. ПРЕДЛАГАЕМОЕ РЕШЕНИЕ

3.1. Влияние порядка обхода ограничений на скорость сходимости

Как было сказано в главе 2.3, одним из недостатков алгоритма PBD является его зависимость от порядка обхода ограничений. При этом, алгоритм XPBD не решает этой проблемы. Авторами статьи [3] предлагается обрабатывать ограничения при помощи алгоритма, основанного на методе Якоби (рис. 2.3), однако на практике данный подход является менее стабильным. Автором данной работы было принято решение изучить влияние порядка обхода на скорость сходимости алгоритма PBD. Стоит отметить, что результаты данного исследования также применимы к алгоритму XPBD, так как в абсолютно нерастяжимой ткани алгоритмы PBD и XPBD совпадают.

Для начала введем понятие “веревка” - тривиальной ткани, характеризующейся тем, что частицы связаны друг с другом ограничениями “пружина” последовательно, как представлено на рис. 3.1.

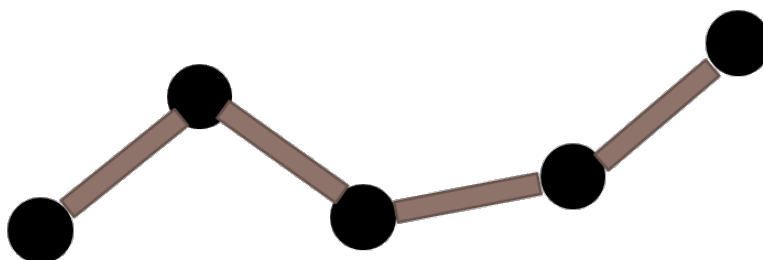


Рис.3.1. Пример веревки. Черным обозначены частицы, коричневым обозначены ограничения

Проверим эмпирически, имеет ли место влияние порядка обхода на сходимость. Для этого поставим следующий эксперимент:

- Рассматривается одномерное пространство.
- В данном пространстве помещается веревка, состоящая из четырёх частиц.
- Крайняя левая частица оттягивается от остальных и закрепляется в пространстве.
- Рассматривается работа первой итерации двух вариантов алгоритма PBD, обрабатывающих ограничения в разном порядке. Первый - слева направо, второй - справа налево.

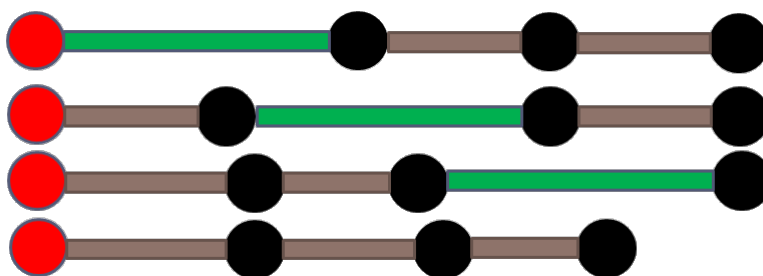


Рис.3.2. Эксперимент с обходом слева направо. Сверху вниз представлены состояния веревки на каждом шаге первой итерации алгоритма PBD. Черным обозначены частицы, коричневым обозначены ограничения. Красным обозначена закрепленная частица, зеленым обозначено ограничение, которое будет обрабатываться следующим.

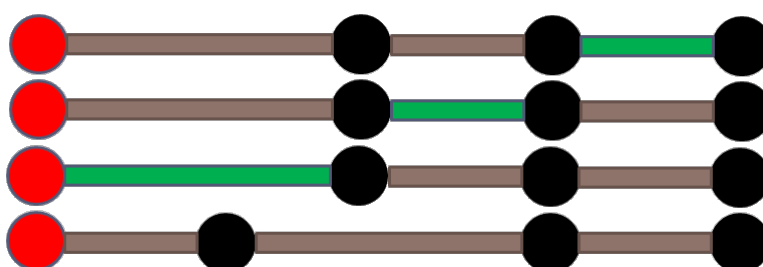


Рис.3.3. Эксперимент с обходом справа налево. Сверху вниз представлены состояния веревки на каждом шаге первой итерации алгоритма PBD. Черным обозначены частицы, коричневым обозначены ограничения. Красным обозначена закрепленная частица, зеленым обозначено ограничение, которое будет обрабатываться следующим.

Из экспериментов, продемонстрированных на рис.3.2 и рис.3.3, можно заметить, что обход слева направо (рис.3.2) позволяет за первую итерацию перенести растяжение с первого ограничения до последнего и убрать его полностью за счет крайней правой вершины. С другой стороны, обход справа налево (рис.3.3) за первую итерацию перенес растяжение только на второе ограничение. Стоит отметить, что результаты данного эксперимента совпадают с результатами, полученными в [4].

Однако, данный эксперимент показывает только влияние двух обходов, и только на первую итерацию алгоритма. В связи с этим, поставим следующий эксперимент:

- Рассматривается одномерное пространство.
- В данном пространстве помещается нерастяжимая веревка, состоящая из N частиц.
- Каждая частица находится на единичном расстоянии относительно соседних частиц.

- Для каждого ограничения расстояние покоя принимается равным единице.
- Крайняя левая частица оттягивается от остальных частиц и закрепляется в пространстве таким образом, чтобы расстояние между ней и соседней частицей стало равно двум.
- Рассматриваются следующие порядки обхода:
 1. *forward* - обход слева направо.
 2. *backward* - обход справа налево.
 3. *shuffle* - каждая четная итерация запускается по обходу слева направо, каждая нечетная итерация запускается по обходу справа налево.
 4. *random* - на каждой итерации список ограничений перемешивается и обрабатывается в случайном порядке.
 5. *coloring* - сначала обрабатываются ограничения с четными номерами, затем ограничения с нечетными номерами.
- Запускается алгоритм PBD с различными наборами параметров, и подсчитывается, сколько итераций потребовалось, для того чтобы веревка была просимулирована.
- Вевка считается просимулированной, если сумма модулей значений функций ограничений стала меньше значения погрешности, принятого равным 10^{-5} .

Данный эксперимент был поставлен с использованием языка программирования Python. Результаты данного эксперимента приведены в табл. 3.1 и на рис. 3.4.

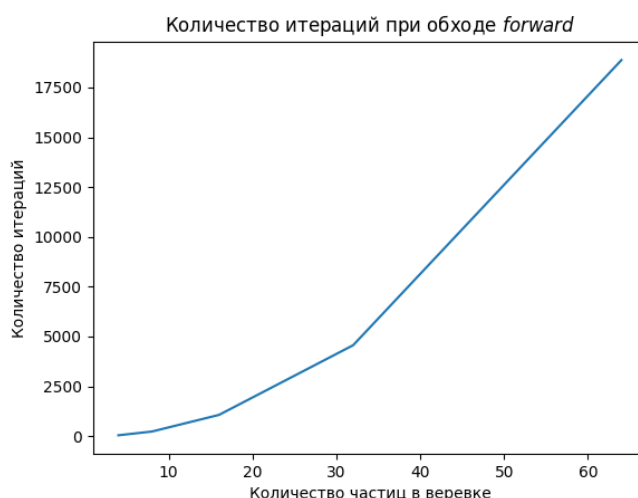


Рис.3.4. Зависимость числа итераций от длины веревки в эксперименте для веревки в одномерном пространстве. Обход *forward*.

Таблица 3.1

Результаты эксперимента применения различных обходов для различных веревек в одномерном пространстве

N	Порядок обхода	Количество итераций
4	<i>forward</i>	41
4	<i>backward</i>	43
4	<i>shuffle</i>	63
4	<i>random</i>	51
4	<i>coloring</i>	42
8	<i>forward</i>	229
8	<i>backward</i>	235
8	<i>shuffle</i>	278
8	<i>random</i>	293
8	<i>coloring</i>	232
16	<i>forward</i>	1064
16	<i>backward</i>	1078
16	<i>shuffle</i>	1156
16	<i>random</i>	1353
16	<i>coloring</i>	1071
32	<i>forward</i>	4562
32	<i>backward</i>	4592
32	<i>shuffle</i>	4738
32	<i>random</i>	5749
32	<i>coloring</i>	4577
64	<i>forward</i>	18876
64	<i>backward</i>	18938
64	<i>shuffle</i>	19222
64	<i>random</i>	23746
64	<i>coloring</i>	18907

На основании полученных данных можно сделать следующие выводы:

1. Порядок обхода влияет на количество итераций
2. Количество частиц в верёвке влияет на количество итераций
3. Наблюдается квадратичная зависимость количества итераций от количества частиц
4. Порядок обхода влияет на количество итераций много меньше, чем количество частиц
5. Наилучшим с точки зрения количества итераций является обход *forward*, что соответствует наблюдению в работе [4]
6. Вторым по количеству итераций является обход *coloring*

Стоит отметить, что обход *forward*, несмотря на то, что является оптимальным по количеству итераций, не применим в параллельной реализации. В свою очередь, обход *coloring*, основанный на принципе реберной раскраски графа, может быть применён в параллельном алгоритме, так как, в силу определения реберной раскраски, никакие два ребра одного цвета не имеют общих вершин. Это означает, что можно параллельно обрабатывать все ограничения, соответствующие ребрам одного цвета, тем самым уменьшив время выполнения одной итерации.

3.2. Эвристический алгоритм для улучшения сходимости

Рассмотрим эксперимент, приведенный в табл. 3.1, и представим (в виде графика) отношение суммы модулей функций ограничений к номеру итерации (рис. 3.5).

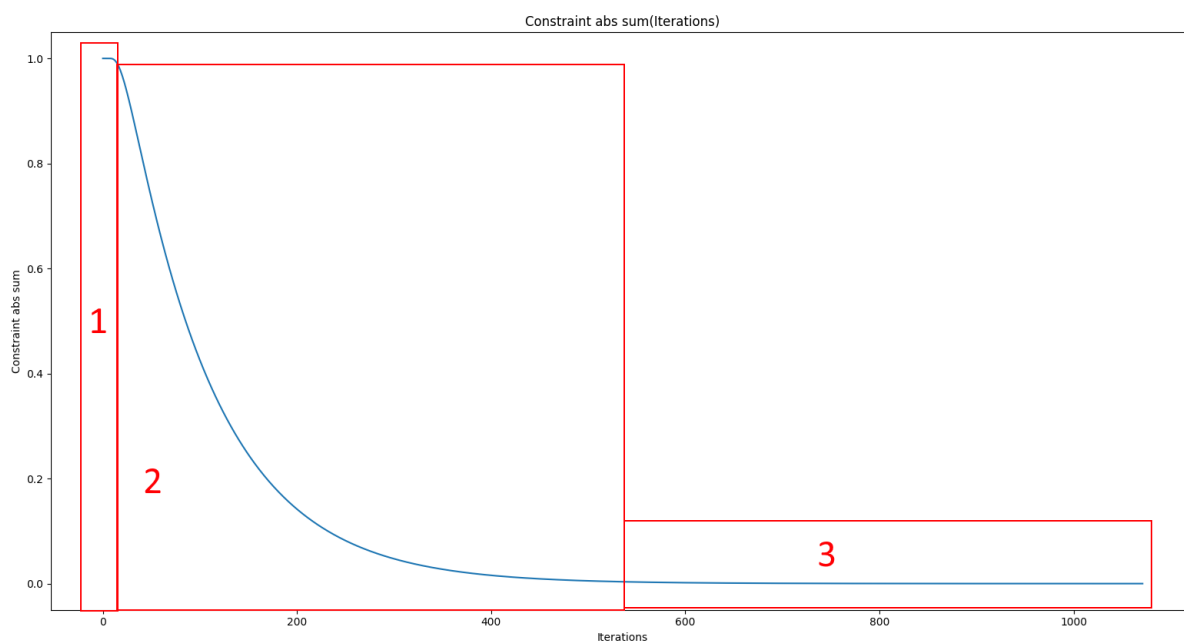


Рис.3.5. Отношение суммы модулей функций ограничений к номеру итерации. Количество частиц 16, обход *coloring*.

На данном графике можно выделить три сегмента:

1. Небольшое плато в начале графика. Размер данной секции и является является различающимся при использовании обходов *forward*, *backward* и *coloring*.
2. Резкий спад. На данном этапе “снимается” наибольшее растяжение.
3. Большое плато, в котором достигается необходимая точность.

Исходя из данного графика, можно заметить, что наибольшее число итераций занимают именно секции 2 и 3. Рассмотрим (табл. 3.2) значения функций ограничений на некоторых итерациях алгоритма PBD, использующего обход *forward*, так как для него секция 1 отсутствует.

Таблица 3.2

Значения функций ограничений для одномерной веревки (8 частиц) на различных итерациях алгоритма PBD обходе *forward*

Итер.	$\sum_{i=1}^7 C_i $	$ C_1 $	$ C_2 $	$ C_3 $	$ C_4 $	$ C_5 $	$ C_6 $	$ C_7 $
0	1.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0
1	0.984375	0.5	0.25	0.125	0.0625	0.03125	0.015625	0.0
2	0.957031	0.375	0.25	0.15625	0.09375	0.054687	0.027343	0.0
3	0.922851	0.3125	0.234375	0.1640625	0.109375	0.068359	0.034179	0.0
4	0.885253	0.273437	0.21875	0.164062	0.116210	0.075195	0.037597	0.0
5	0.846374	0.246093	0.205078	0.160644	0.117919	0.077758	0.038879	0.0
10	0.662745	0.174022	0.155377	0.130353	0.100749	0.068161	0.034081	0.0
20	0.399291	0.103489	0.093230	0.078865	0.061312	0.041595	0.020797	0.0
50	0.087025	0.022551	0.020318	0.017189	0.013364	0.009067	0.004533	0.0
100	0.006869	0.001780	0.001603	0.001356	0.001054	0.000715	0.000357	0.0
200	0.000042	0.000011	0.000009	0.000008	0.000006	0.000004	0.000002	0.0
229	0.000009	0.000002	0.000002	0.000001	0.000001	0.000001	0.000001	0.0

Исходя из полученных значений, заметим:

1. После первой итерации значения модулей функций ограничения совпадают с элементами последовательности $\{\frac{1}{2^n}\}$.
2. После первой итерации суммарное значение модулей функций ограничения уменьшилось на $\frac{1}{64} = \frac{1}{2^8}$.
3. После каждой итерации значение модуля функции седьмого ограничения всегда равно 0.0.
4. Наибольшая скорость сходимости наблюдается тогда, когда значения модулей ограничения близки друг к другу.

В связи с наблюдением 4 возникает желание “разровнять” значения функций растяжения между собой так, чтобы увеличить скорость сходимости. Для этого, попробуем ещё раз запустить этот эксперимент, однако раз в несколько итераций (обозначим это количество *HeuristicStep*), помимо обхода ограничений будем запускать алгоритм, представленный на рис. 3.6.

Algorithm

Input: количество частиц N , предсказанные положения частиц p_i^* ,
параметр интерполяции $HeuristicAlpha$

Output: новые положения частиц p_i^*

1. $ropeLength = |p_N^* - p_1^*|;$
2. $ropeDir = \frac{p_N^* - p_1^*}{ropeLength};$
3. **for** $i \in 1..N$ **do**
4. $lengthFromStart = \frac{i-1}{N-1} * ropeLength;$
5. $p'_i = p_1^* + ropeDir * lengthFromStart;$
6. $p_i^* = p_i^* * (1 - HeuristicAlpha) + p'_i * (HeuristicAlpha);$

Рис.3.6. Псевдокод эвристического алгоритма для веревки в одномерном пространстве

На рис.3.7 и рис.3.8 представлено отношение суммарного значения модулей функций ограничения к номеру итерации для разных значений $HeuristicStep$ и $HeuristicAlpha$.

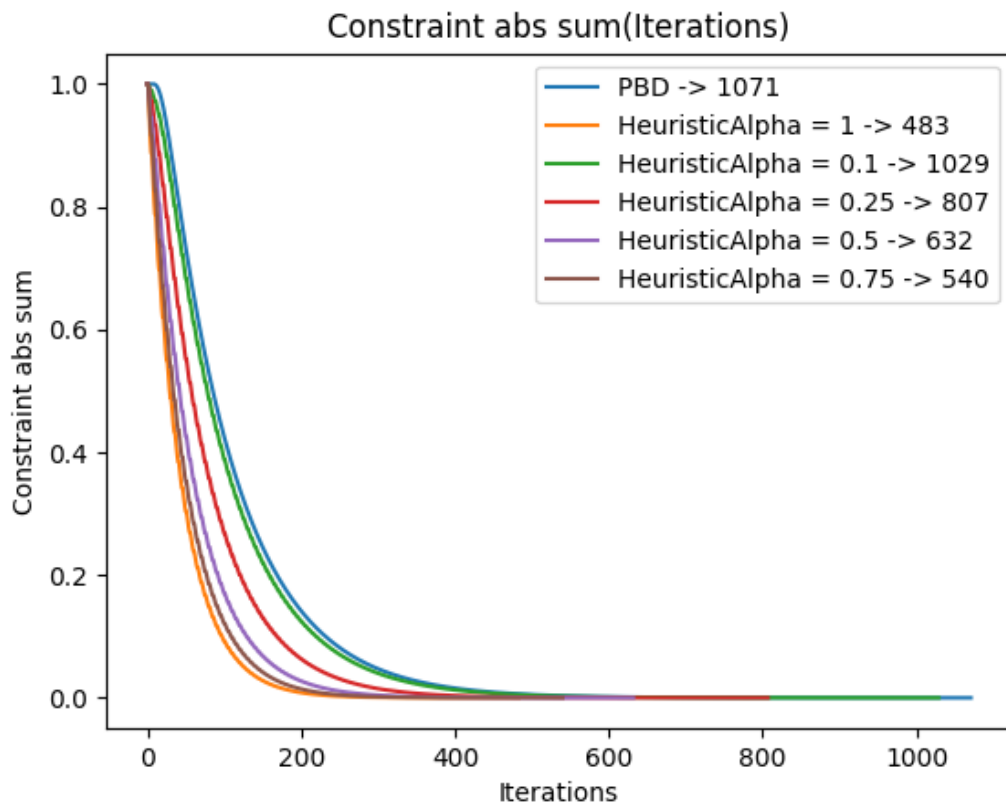


Рис.3.7. Отношение суммы модулей функций ограничений к номеру итерации с использованием эвристического алгоритма. Количество частиц 16, обход *coloring*, $HeuristicStep = 5$. В легенде обозначено итоговое количество итераций

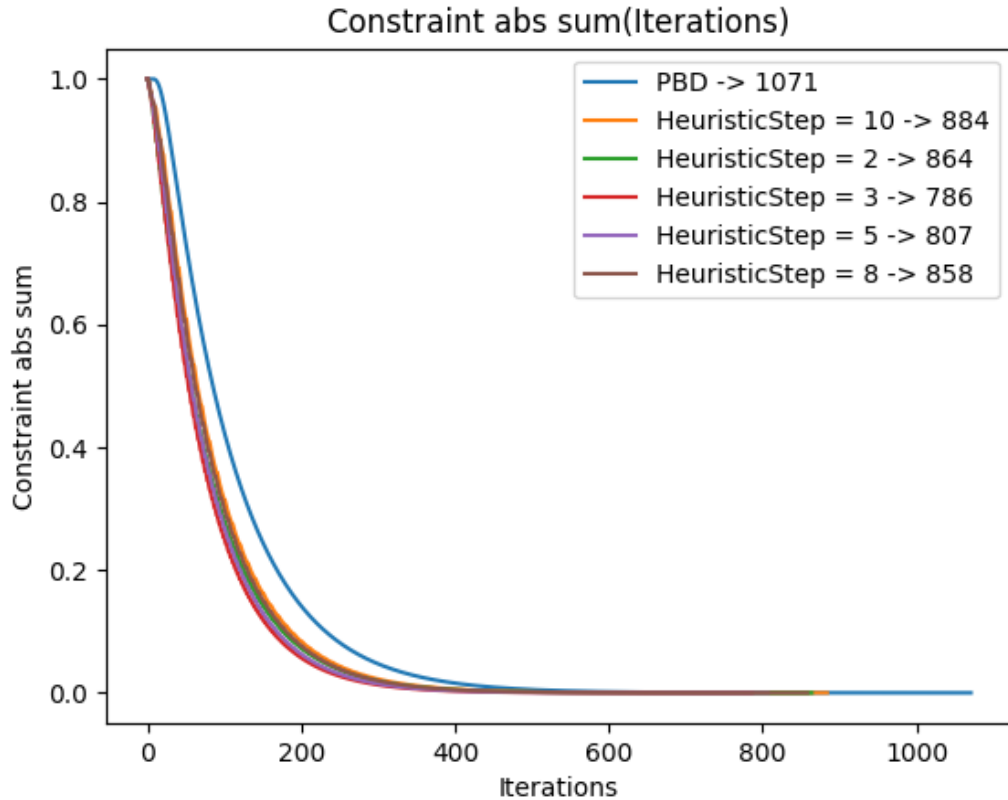


Рис.3.8. Отношение суммы модулей функций ограничений к номеру итерации с использованием эвристического алгоритма. Количество частиц 16, обход *coloring*, *HeuristicAlpha* = 0.25. В легенде обозначено итоговое количество итераций

Таким образом, можно сделать вывод, что предлагаемый алгоритм работает, и количество итераций действительно снизилось. Данный алгоритм можно рассматривать как шаг, направленный на удовлетворение условия [3.1](#), которое напрямую вытекает из [2.1](#) и означает равенство “плотностей”.

$$\forall i,j : C_i(p) = C_j(p) = \frac{\sum_k^M C_k(p)}{M} \quad (3.1)$$

Однако, данный алгоритм имеет ряд недостатков:

1. Он (рис.[3.6](#)) применим только в одномерном пространстве.
2. Он применим только для алгоритма PBD.
3. Он применим только если параметр жёсткости равен бесконечности.
4. Он модифицирует положения частиц без использования значений ограничений, связывающих их. Из-за этого на этапе вычисления скоростей частиц будет внесена ошибка, приводящая к заметным колебаниям симулируемой ткани.
5. Он применим только для веревок.

Для решения недостатка под пунктом 1, перестроим алгоритм так, чтобы вместо длин в пространстве он оперировал длинами вдоль веревки. Для этого нам будет необходимо посчитать префиксную сумму длин для веревки. Затем, найти *lengthFromStart* - расстояние (вдоль веревки) от первой частицы до p'_i . Далее, при помощи префиксной суммы можно определить, между какими двумя частицами p_j^* и p_{j+1}^* находится p'_i . И наконец, найти p'_i при помощи линейной интерполяции.

На рис. 3.9 представлено расположение точек при использовании такого подхода.

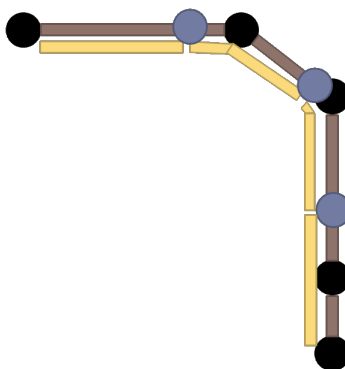


Рис.3.9. Графическое представление работы эвристического алгоритма. Черным представлены точки p_i^* , коричневым - ограничения их соединяющие. Синим представлены точки p'_i используемые в алгоритме, а желтым показаны секции с равной длиной вдоль веревки

Описанный эвристический алгоритм можно также рассматривать как построение кривой по опорным точкам, с последующим выбором новых опорных точек. Результатом данных преобразований, которые можно назвать “ресемплингом”, не всегда будет та же самая кривая (как показано на рис. 3.10), однако, используя параметры *HeuristicAlpha* и *HeuristicStep*, можно контролировать величину допускаемого отклонения. Помимо этого, стоит отметить, что при большой плотности частиц такие ситуации маловероятны.

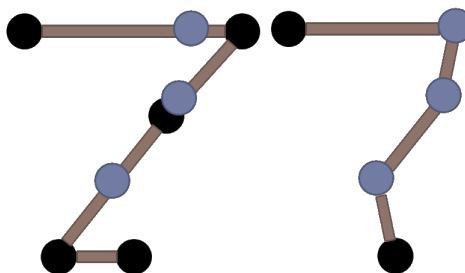


Рис.3.10. Графическое представление ошибок эвристического алгоритма при *HeuristicAlpha* = 1. Слева представлена структура верёвки до применения эвристического алгоритма, справа - после. Черным представлены точки p_i^* , коричневым - ограничения их соединяющие. Синим представлены точки p'_i получаемые в результате применения алгоритма.

Для решения недостатка, связанного с применимостью только для алгоритма PBD, а также бесконечной жесткости, перепишем выражение [2.15](#) в форме, аналогичной выражению [3.1](#). Тогда мы получим:

$$\forall i, j : C_i(p) + \frac{\alpha}{\Delta t^2} \lambda_i = C_j(p) + \frac{\alpha}{\Delta t^2} \lambda_j = \frac{\sum_k^M C_k(p) + \frac{\alpha}{\Delta t^2} \lambda_k}{M} \quad (3.2)$$

В данном выражении λ_k является неизвестной величиной, однако, используя подход Small steps, описанный в параграфе [2.3](#), мы можем приравнять λ_k к $\Delta \lambda_k$, которые можем найти при помощи формулы [2.20](#). Используя в данном алгоритме вместо суммы длин, сумму величин приведенных в [3.2](#), мы делаем возможным применение эвристического алгоритма совместно с XPBD, а также при различных значениях параметра жесткости.

Для решения недостатка, связанного с некорректным вычислением скоростей при выполнении следующей итерации, необходимо, воспользовавшись техникой Small Steps, вынести применение эвристического алгоритма из цикла обработки ограничений, поместив его перед интегрированием скоростей. Затем, модифицировать значения положений частиц, полученные в течение прошлой итерации, таким образом, чтобы скорости проинтерполировались соответственно проинтерполированным значениям (всё также используя идею “ресемплинга”).

Для решения последней проблемы, связанной с тем, что данный метод применим только к веревкам, вспомним (глава [2.4](#)), что основным подходом для расставления ограничений является задание поверхности ткани в виде прямоугольной сетки. Тогда рассмотрим линии, образующие эту сетку, получающиеся при соединении частиц горизонтальными и вертикальными структурными ограничениями. Данные линии являются веревками, а значит к ним также применим эвристический метод. При этом стоит отметить, что веревки одной ориентации могут быть обработаны параллельно между собой (рис. [3.11](#)).

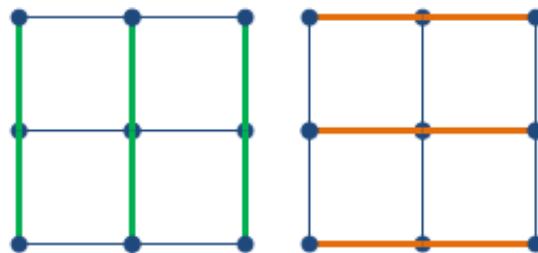


Рис.3.11. Визуальная демонстрация расположения веревок для ткани размерности 3 на 3 частицы

Применив все вышеописанные улучшения, мы получаем эвристический алгоритм представленный на рис.3.12.

Algorithm *ImprovedHeuristicAlgorithm*

- Input:** время шага симуляции Δt , текущие положения частиц $p_i(t)$, предыдущие положения частиц $p_i(t - \Delta t)$, обратные массы частиц im_i , обратная жесткость α , параметр *HeuristicAlpha*
- Output:** обновленные положения частиц $p_i(t)$, $p_i(t - \Delta t)$
1. Заполнение массива *lengths* последовательными значениями частичных сумм ряда $|p_i(t) - p_{i+1}(t)|$;
 2. Заполнение массива *coeffs* последовательными значениями частичных сумм ряда $C_i(p_i(t), p_{i+1}(t)) + \Delta\lambda_i$;
 3. $totalLength = lengths_{N-1}$;
 4. $totalCoeff = coeffs_{N-1}$;
 5. **for** $i \in 2..N - 1$ **do**
 6. $lengthFromStart = totalLength * coeffs_i / totalCoeff$;
 7. Бинарный поиск $j : lengths_{j-1} \leq lengthFromStart < lengths_j$;
 8. $t = \frac{lengthFromStart - lengths_{j-1}}{lengths_j - lengths_{j-1}}$;
 9. $p'_i = p_{j-1}(t) * (1 - t) + p_j(t) * t$;
 10. $v'_i = \frac{p_{j-1}(t) - p_{j-1}(t - \Delta t)}{\Delta t} * (1 - t) + \frac{p_j(t) - p_j(t - \Delta t)}{\Delta t} * t$;
 11. $v_i = \frac{p_i(t) - p_i(t - \Delta t)}{\Delta t}$;
 12. $p_i(t) = p_i(t) * (1 - HeuristicAlpha) + p'_i * (HeuristicAlpha)$;
 13. $v_i = v_i * (1 - HeuristicAlpha) + v'_i * (HeuristicAlpha)$;
 14. $p_i(t - \Delta t) = p_i(t) - v_i * \Delta t$;

Рис.3.12. Псевдокод эвристического алгоритма для веревки в многомерном пространстве с поддержкой XPBD и обработкой скоростей

А соответствующий ему алгоритм Extended Position Based Dynamics, в котором применена техника Small Steps, и показано, как именно встраивается в его работу эвристический алгоритм, представлен на рис.3.13

Algorithm

Input: время шага симуляции Δt , количество шагов $solverStep$, количество итераций $solverIteration$, текущее состояние частиц, предыдущее состояние частиц, ограничения, функция суммы внешних сил $f_{ext}(p)$, параметр $HeuristicStep$, параметр $HeuristicAlpha$

Output: положение частиц спустя заданное время $p_i(t + \Delta t)$

1. $stepT = \Delta t / solverStep$;
2. **for** $step \in 1..solverStep$ **do**
3. **if** $mod(step, HeuristicStep) = 0$ **then**
4. **for** *horizontal rope* **do** $ImprovedHeuristicAlgorithm()$;
5. **for** *vertical rope* **do** $ImprovedHeuristicAlgorithm()$;
6. Произвести шаг XPBD в соответствии с рис. 2.15

Рис.3.13. Псевдокод алгоритма Extended Position Based Dynamics с использованием эвристического алгоритма

3.3. Оптимизация параллельного обхода ограничений для GPU

В рамках данной работы, перед автором стояла задача не только разработки эвристического алгоритма, описанного в параграфе 3.2, но и реализации с использованием GPU. Действительно, главной отличительной особенностью GPU относительно CPU является много большее количество вычислительных ядер. Таким образом, учитывая закон Амдала, некоторые алгоритмы могут быть многократно ускорены за счет грамотного разбиения на подзадачи. Лучше всего GPU справляется в ситуации, когда вышеупомянутые подзадачи являются вычислительными задачами имеющими одинаковый алгоритм решения, однако разные входные данные. Подход, в котором GPU используется для вычисления задач общего назначения, не связанных с отображением графики, принято называть GPGPU - General-purpose computing on graphics processing units. Для реализации данного подхода есть два пути: либо использование специализированных библиотек для GPGPU, таких как Nvidia CUDA или OpenCL, либо использование вычислительных шейдеров в графических библиотеках, таких как DirectX12 или Vulkan. В рамках данной работы, автором рассматривается применение вычислительных шейдеров, так как конечной целью является встраивание симуляции тканей в приложение, использующее DirectX12.

Введем некоторые основные понятия программирования в парадигме GPGPU. Если говорить с точки зрения выполнения, то на GPU запускаются шейдеры. Шейдеры запускаются параллельно, имеют одинаковый код, но получают различающийся входной аргумент - номер соответствующего потока исполнения. Потоки объединяются в группы, причем все группы имеют одинаковый размер. Программист же, при отправке команды на запуск какого-либо шейдера, указывает целое количество групп, которые он хочет выполнить. Для запуска каждой группы GPU выделяет целое число *Wavefront*-ов - наборов из процессоров, которые выполняют операции абсолютно параллельно. В случае использования видеокарт Nvidia размер одного *Wavefront*-а равен 32, а в случае AMD - 64.

Если говорить с точки зрения памяти, каждый поток имеет локальную память (для локальных переменных шейдера), глобальную память (для считывания входных параметров и записи результирующих) и *groupshared* память. Последняя является более быстрой по времени чтения-записи по отношению к глобальной, однако выделяется только в рамках запуска одной группы и может быть использована потоками внутри группы для обмена данными.

Если же говорить с точки зрения синхронизации, то в рамках языка HLSL и DirectX12, есть два способа синхронизации вычислительных шейдеров:

1. Применение барьера глобальной памяти, в которую записывается результат. Таким образом, следующие команды не начнут выполняться до тех пор, пока каждая группа не закончит записывать в данную память.
2. Использование команды шейдера *GroupMemoryBarrierWithGroupSync*, которая позволяет синхронизировать все *Wavefront*-ы внутри одной группы.

Рассмотрим алгоритм XPBD, представленный на рис. 3.13, и перепишем его в виде, подходящем для параллельного выполнения (рис. 3.14).

Как можно заметить, данный алгоритм отлично разбивается на подзадачи, имеющие одинаковый алгоритм, но разные входные данные, что прекрасно лоскуется с использованием технологии GPGPU. Однако возникает вопрос: как именно реализовать данный алгоритм для выполнения при помощи GPU.

Первый вариант: представить весь этот алгоритм как единый шейдер, для которого будет запущена только одна группа (такое решение используется в NVidia Cloth). Тогда каждый *syncPoint* будет представлен как вызов *GroupMemoryBarrierWithGroupSync*, а частицы и ограничения сначала должны быть загружены в *groupshared* память, а после окончания работы данного

Algorithm

```

1.  for step ∈ 1..solverStep do
2.      if mod(step, HeuristicStep) = 0 then
3.          for getHorRopes() do in parallel
4.              ImprovedHeuristicAlgorithm()
5.          syncPoint();
6.          for getVerRopes() do in parallel
7.              ImprovedHeuristicAlgorithm()
8.      for getParticles() do in parallel predictParticle() ;
9.      syncPoint();
10.     for getParticles() do in parallel genCollisionConstraint() ;
11.     syncPoint();
12.     for type ∈ {structural, shear, bending} do
13.         for color ∈ getColors(type) do
14.             for getConstraints(type, color) do in parallel
15.                 solveXPBDConstraint();
16.             syncPoint();
17.     for getCollisionConstraints() do in parallel
18.         solveCollisionConstaint();
19.     syncPoint();
20.     for getParticles() do in parallel saveNewParticlePos() ;
21.     syncPoint();

```

Рис.3.14. Псевдокод алгоритма Extended Position Based Dynamics с использованием Small steps, эвристического алгоритма и отмеченными местами возможной параллелизации

алгоритма выгружены обратно в глобальную. Недостатком данного подхода является то, что размер группы в языке HLSL не может превышать 1024, а значит невозможно использовать более 1024 потоков.

Второй вариант: представить каждый “**for * do in parallel**” как отдельный шейдер. Тогда каждый *syncPoint* будет представлен как выставление барьера. Недостатком данного подхода является то, что не используется *groupshared* память, тем самым алгоритм теряет в производительности из-за многочисленных

записей и чтений из глобальной памяти. Также, недостатком является частая смена и вызов различных шейдеров, что приводит к дополнительным расходам по времени выполнения.

В рамках исследования алгоритма XPBD с использованием техники Small steps и обходом ограничений на основании реберной раскраски было замечено, что в течение одной итерации влияние частиц ограничено. Это означает, что для подсчета положения конкретной частицы на следующей итерации необходимо знать положение лишь некоторых соседних частиц, а не всей ткани.

Для определения границ влияния частиц был поставлен следующий эксперимент:

1. Строится сетка размером $(N + 2)$ на $(N + 2)$ вершины, соединенные между собой аналогично тому, как частицы ткани соединены структурными ограничениями.
2. Внешние вершины сетки отмечаются красным цветом, в то время как внутренние ограничения помечаются синим.
3. В том же порядке, в котором обрабатываются структурные ограничения в ткани, обрабатываем ребра в сетке по следующим образом: если одна из инцидентных вершин красная, то перекрашиваем вторую вершину в красный цвет.

Пример данного эксперимента можно увидеть на рис. [3.15](#).

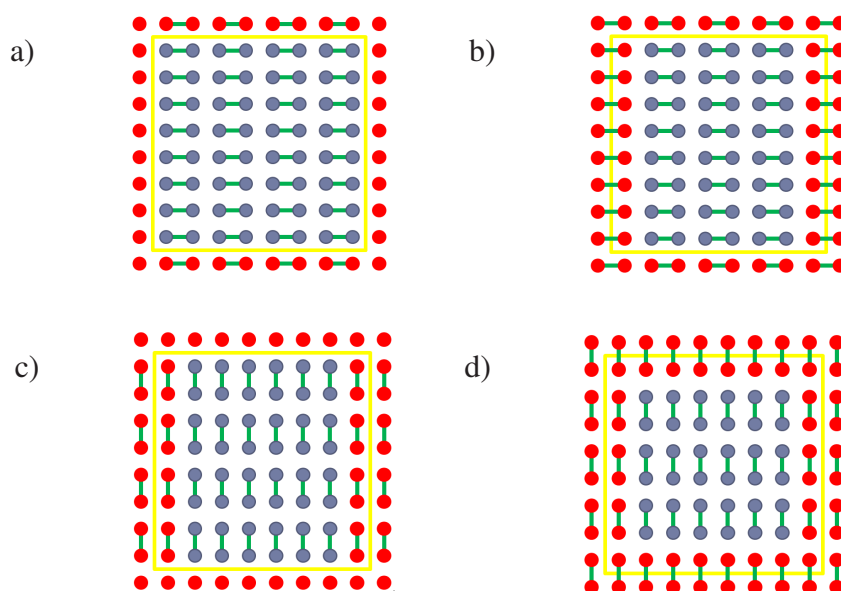


Рис.3.15. Визуальная демонстрация шагов эксперимента по определению границ влияния частиц, связанных структурными ограничениями, при $N = 8$. Кругами обозначены вершины, зеленым обозначены обрабатываемые ребра, желтым обозначена граница подсетки $N \times N$.

В результате проведения данного эксперимента, синим остаются обозначены те вершины, для вычисления положений которых достаточно только положения вершин, лежащих внутри желтой границы. Таким образом, можно разбить всю сетку ткани на пересекающиеся области (назовем их “заплатками”) и обсчитывать каждую “заплатку” в отдельной группе. При этом, подберем размер заплатки N и области пересечения таким образом, чтобы частицы, соответствующие синим вершинам, полностью покрывали поверхность ткани. Тогда псевдокод шейдера для обработки структурных ограничений для “заплатки” будет выглядеть так, как показано на рис. 3.16.

Algorithm

1. Параллельно считать в groupshared память частицы заплатки N на N ;
2. *syncPoint()*;
3. **for** $color \in getColors(structural)$ **do**
4. **for** $getConstraints(structural, color)$ **do in parallel**
5. *solveXPBDConstraint()*;
6. *syncPoint()*;
7. Параллельно записать в глобальную память посчитанные частицы соответствующие синим вершинам;

Рис.3.16. Псевдокод шейдера для обработки структурных ограничений.

Исходя из значений, представленных на табл. 3.3, было принято решение использовать $N = 10$, а на каждую “заплатку” выделять группу из 64 потоков. Благодаря этому, достигается баланс между временем работы *GroupMemoryBarrierWithGroupSync* и коэффициентом полезного действия (отношение синих вершин к общему числу вершин).

Таблица 3.3

Результаты эксперимента при разных значениях N размера подсетки $N \times N$

N	Кол-во синих	Кол-во красных	Макс. пар. ограничений
2	0	4	2
4	4	12	8
6	16	20	18
8	36	28	32
10	64	36	50
12	100	44	72
14	144	52	98

Данная идея обсчета ограничений “заплатами” может быть также применена для ограничений сдвига и ограничений изгиба и позволяет объединить в себе преимущества двух подходов представления алгоритма (рис. 3.14). С одной стороны, количество используемых потоков неограничено, так как каждая группа будет обсчитывать свою “заплатку”. С другой стороны, использование *groupshared* памяти для сохранения промежуточного результата между обработкой групп ограничений одного типа, соответствующих разным цветам рёберной раскраски, позволяет существенно уменьшить количество обращений к глобальной памяти. Помимо этого, данный подход уменьшает число переключений и вызовов шейдеров.

3.4. Алгоритм нахождения самопересечений

Как было сказано ранее в параграфе 2.2, авторами оригинальной статьи уже был предложен алгоритм обнаружения самопересечений, однако построение сложной структуры, представленной в [13], является слишком долгой операцией для выполнения на GPU, особенно для систем симуляции реального времени. В связи с этим, был разработан и реализован другой алгоритм обнаружения самопересечений.

Во-первых, вместо обработки пересечений частиц и треугольников, было принято решение реализовать обработку пересечений двух частиц на основании того, что каждая частица задана сферой. При этом, радиусы упомянутых сфер выбираются таким образом, чтобы они были не меньше, чем половина длины покоя структурного ограничения. Помимо этого, самопересечения между частицами, непосредственно соединенными структурными ограничениями или ограничениями сдвига - не проверяются, так как их положение уже задается при помощи ограничений.

Во-вторых, для обнаружения соседних частиц используется алгоритм пространственного хеширования [11], в котором трехмерное пространство разбивается на ячейки, и номера этих ячеек пространства выступают в качестве ключа. Идея данного алгоритма заключается в создании двух массивов: первый (*Pointers*) будет хранить индексы частиц в глобальном списке частиц, а второй (*HashMap*) будет для каждой ячейки пространства хранить индекс в массиве *Pointers*, соответствующий первому из элементов, лежащих в данной ячейке пространства. Таким образом, для того, чтобы перебрать все элементы, принадлежащие данной ячейке пространства, достаточно получить индекс ячейки пространства в массиве

HashMap (назовем этот индекс *hash*), а затем рассмотреть все элементы массива *Pointers* начиная с элемента под номером *HashMap[hash]* (включительно) и до элемента *HashMap[hash + 1]* (не включительно). Преимуществом данного алгоритма хеширования является то, что для построения данной хеш-таблицы не требуется сортировок, или других задач, параллельное решение которых невозможно или не имеет смысла, вместо них в данном алгоритме применяется лишь префиксная сумма, для которой уже разработаны алгоритмы параллельного вычисления (например представленный в [5]). Недостатком данной хеш-таблицы является то, что при её использовании отсутствует возможность динамического добавления и удаления, однако в рамках поставленной задачи это не требуется.

В третьих, вместо того, чтобы записывать в хеш-таблицу только частицы, расположенные в соответствующей ячейке пространства, было принято решение дополнительно записывать ещё и частицы, расположенные в соседних ячейках. Данное изменение не влияет на объем памяти, занимаемый хеш-таблицей, но позволяет существенно уменьшить количество обращений к глобальной памяти на этапе обработки соседних вершин.

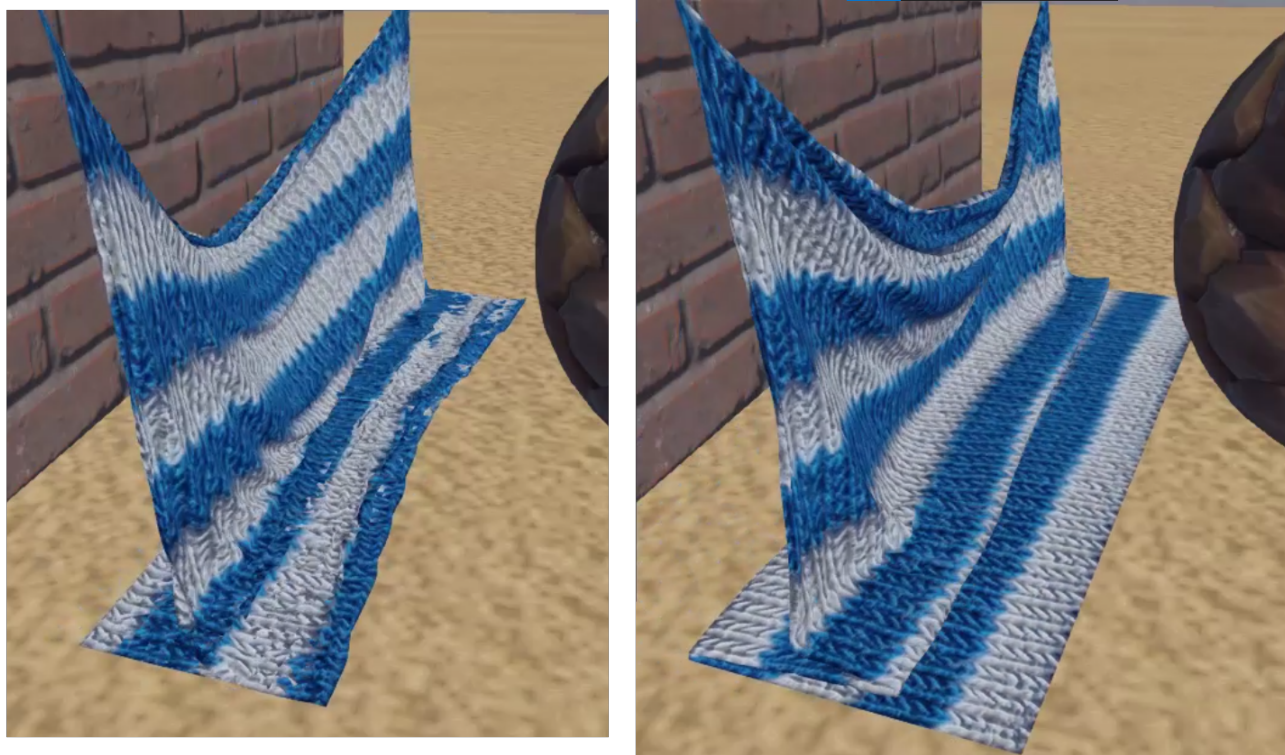


Рис.3.17. Сравнение поведения симулируемой ткани с применением алгоритма самопересечения и без него. На рисунке слева проверка самопересечений не производится, а на рисунке справа используется описанный алгоритм.

3.5. Визуальное представление

Результатом вышеприведенных алгоритмов симуляции поверхности ткани является множество точек, не подходящее для визуального представления. В связи с этим возникает вопрос о том, как правильно отображать результаты работы проделанной симуляции. Назовем симуляционным мешем (simulation mesh) - множество точек, соединенных ограничениями, являющееся результатом работы симуляционного алгоритма. Мешем отрисовки (render mesh) назовем набор данных, используемых для отрисовки некоторого объекта с помощью графических систем. Меш отрисовки может быть получен одним из двух способов:

1. Взят из готовой трёхмерной модели, созданной в геометрическом редакторе. Тогда часть вершин меша отрисовки закрепляется за частицами симуляционного меша и, подобно алгоритму скелетной анимации, симуляционный меш становится “скелетом” для меша отрисовки. (рис. 3.18).
2. Процедурно сгенерирован на основании симуляционного меша. Тогда он может иметь более простую структуру и будет задаваться какими-либо правилами.

В рамках данной работы была разработана система процедурной генерации меша отрисовки, поддерживаемого системой визуализации "DX12Engine" [21]. Модель освещения, используемая в данной системе, относится к основанным на физической модели микрограней [10], данная модель освещения также подразумевает сохранение энергии в соответствии с “основным уравнением рендеринга” [7]. Для этого используется двулучевая функция отражательной способности Кука-Торренса [1] и поправка Френеля-Шлика [16]. Для рассеянных отражений применяется модель Ламберта [6], а глобальное освещение производится по методу Image-Based Lighting [2].

Чтобы построить меш отрисовки, который может быть отображен с использованием вышеописанной модели освещения, требуется задать следующие параметры:

- Положения вершин меша отрисовки.
- Индексы вершин, образующих треугольники, для задания геометрической структуры.
- Вектор нормали в каждой вершине меша.
- Текстурные координаты в каждой вершине меша.
- Тангент вектор в каждой вершине меша.

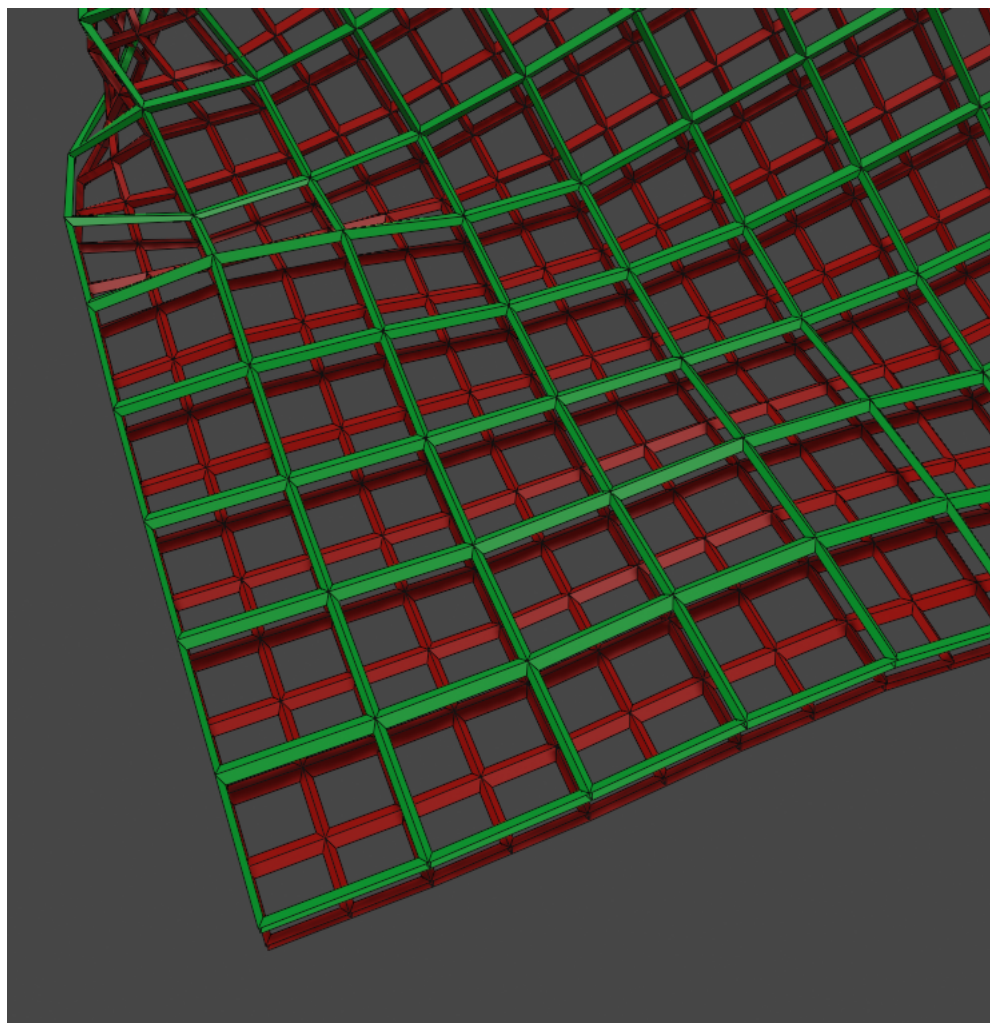


Рис.3.18. Пример закрепления меша отрисовки за симуляционным мешем. Зеленым отмечен симуляционный меш, красным отмечен меш отрисовки

- Карта нормалей.
- Коэффициент металличности.
- Коэффициент шероховатости.
- Коэффициент цвета.

Для задания положений вершин меша отрисовки используются вершины симуляционного меша. В таком случае не требуется использовать какой-либо алгоритм для задания положений вершин, не имеющих соответствия среди симуляционных вершин, так как все вершины меша отрисовки будут закреплены за вершинами симуляционного меша.

Для задания индексов вершин используется тот факт, что частицы симуляционного меша представляются в виде прямоугольной сетки (как рассказано в параграфе [2.4](#)). Таким образом, используя структурные ограничения и ограничения сдвига, можно построить два вида сетки, представленные на рис. [3.19](#).

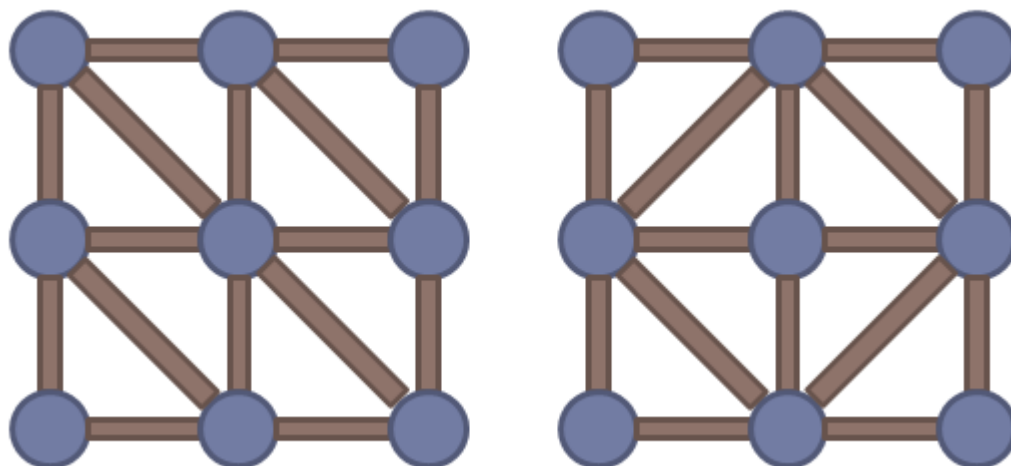


Рис.3.19. Примеры двух вариантов соединения точек меша отрисовки треугольниками.

Для задания вектора нормали в вершине используется алгоритм нахождения нормали в триангулированных моделях. Согласно данному алгоритму, нормалью в вершине принято считать нормализованную сумму нормалей всех треугольников, которые задаются в том числе данной вершиной. Нетрудно заметить, что на рис. 3.19, левая конфигурация предполагает вычисление нормалей шести треугольников для определения нормали центральной вершины, в то время как при использовании правой конфигурации будет достаточно всего четырёх.

Для задания текстурных координат можно воспользоваться тем фактом, что меш отрисовки, также как и симуляционный меш, представляет собой прямоугольную сетку. Введём систему координат вдоль этой прямоугольной сетки таким образом, чтобы крайние вершины имели координаты $(0, 0)$, $(1, 0)$, $(0, 1)$, $(1, 1)$. Для каждой вершины из оставшихся определяются текстурные координаты в соответствии с вышеупомянутой системе координат (рис. 3.20).

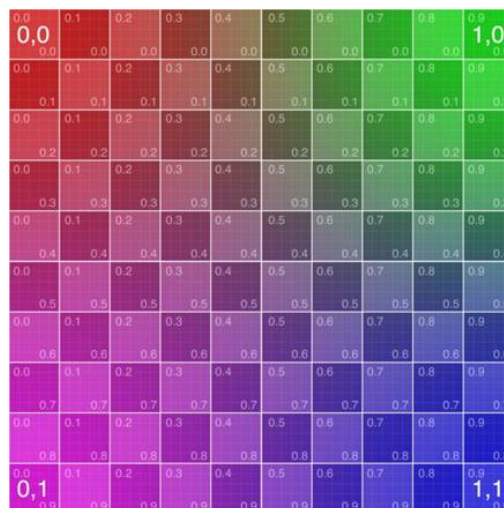


Рис.3.20. Текстурные координаты для поверхности, заданной прямоугольной сеткой

Для задания тангент вектора используются сгенерированные текстурные координаты и векторы нормали в вершинах. Чтобы определить тангент вектор, для каждой вершины меша отрисовки производятся следующие операции:

1. Находится ближайшая “правая” вершина, исходя из представления в текстурных координатах
2. При помощи векторного произведения между нормалью к текущей вершине и направлением на “правую” вершину определяется вектор бинормали
3. Векторное произведение бинормали и нормали в результате определяет тангент вектор.

Все остальные параметры могут быть представлены в виде текстур и найдены в сети Интернет.

Построенный с помощью приведённых выше действий объект готов к использованию в системе визуализации трёхмерной компьютерной графики и может отображать результаты физической симуляции.

ГЛАВА 4. РЕЗУЛЬТАТЫ И ИХ СРАВНИТЕЛЬНЫЙ АНАЛИЗ

Для демонстрации работы предлагаемого решения и сравнения производительности с существующими решениями, в системе визуализации трехмерных сцен “DX12Enigne” была реализована поддержка симуляции и визуализации тканей, с возможностью динамического переключения алгоритма симуляции тканей. В качестве эталонного сравнения рассматривается работа библиотеки NVidia Cloth. Для проведения экспериментов были созданы две тестовые сцены, демонстрирующие различные аспекты поведения симулируемой ткани.

Первая сцена представляет собой открытую городскую местность, в которой размещен флаг, развевающийся на сильном ветру. Движение флага задаётся при помощи системы симуляции тканей. При использовании библиотеки Nvidia Cloth флаг совершает циклические колебания с высокой амплитудой, подобные поведению пружины с грузом под действием силы тяжести (рис. 4.1). При использовании эвристического алгоритма, описанного в параграфе 3.2 подобного поведения не наблюдается (рис. 4.2).



Рис.4.1. Тестовая сцена "Флаг". Используется Nvidia Cloth. Кадры спустя 1, 2 и 3 секунды после запуска симуляции.

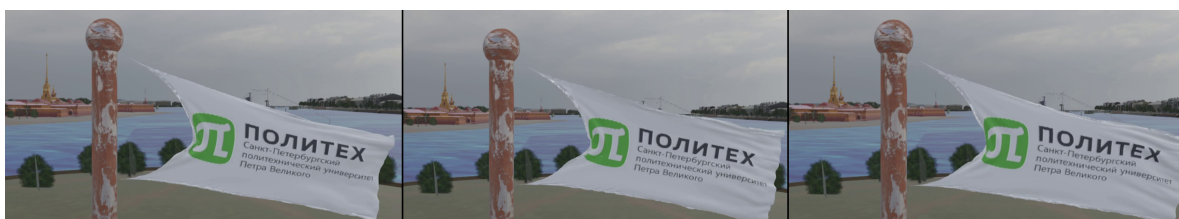


Рис.4.2. Тестовая сцена "Флаг". Используется эвристический алгоритм. Кадры спустя 1, 2 и 3 секунды после запуска симуляции.

Вторая сцена представляет собой учебную аудиторию, в которой находится танцующая девушка. Из окна падает свет солнца, оставляя в помещении динамические тени. Девушка при этом одета в юбку, которая в свою очередь симулируется как поверхность ткани, состоящая из 2048 частиц, 2016 структурных ограничений и 2016 ограничений сдвига. Дополнительно обрабатываются пересечения с ногами и торсом девушки за счет упрощенного представления их как капсул. Также используется алгоритм устранения самопересечений из параграфа 3.4. Алгоритм симуляции использует технику Small steps, разделяя шаг на 30 подшагов. Примеры кадров представлены на рис. 4.3, рис. 4.4, рис. 4.5.



Рис.4.3. Тестовая сцена "Танец". Кадр записанный в начале анимации



Рис.4.4. Тестовая сцена "Танец". Кадр записанный в середине анимации



Рис.4.5. Тестовая сцена "Танец". Кадр записанный в конце анимации

На данной сцене производилось измерение длительности работы алгоритма симуляции тканей в течение всей анимации, для разных алгоритмов симуляции. Результаты представлены на графике (рис. 4.6). Значения “Nvidia Cloth” соответствуют времени работы реализации, используемой в библиотеке Nvidia Cloth на GPU, а значения “Me” соответствуют времени работы разработанной автором реализации алгоритма XPBD с применением всех описанных оптимизаций. Эксперименты проводились на компьютере, характеристики которого представлены в таблице 4.1.

Таблица 4.1

Характеристики компьютера, на котором проводилось тестирование

CPU	Intel Core i5-9600K
GPU	nVidia RTX 2070 SUPER
RAM	32 Gb

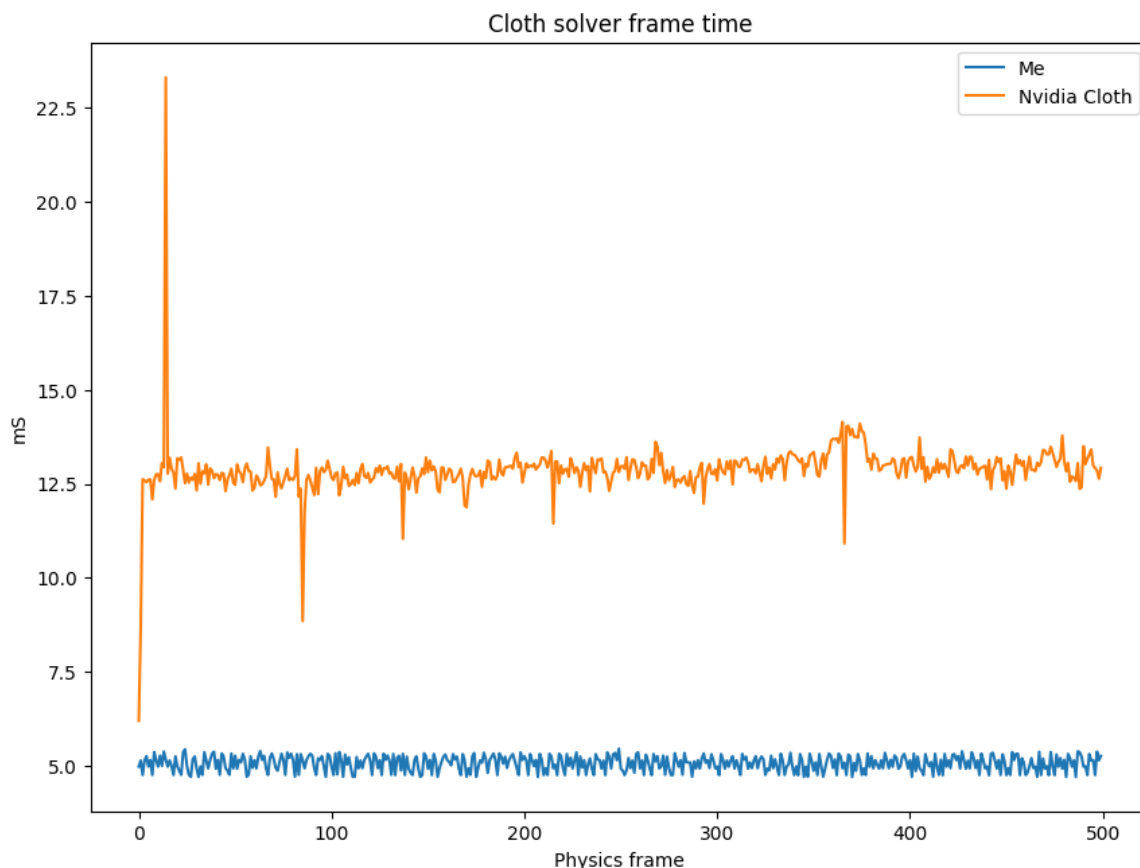


Рис.4.6. График времени работы алгоритма симуляции тканей. По оси X отложен номер кадра анимации, по оси Y - время работы системы симуляции в миллисекундах. Среднее время работы Nvidia Cloth равно 12.87ms, среднее время работы предлагаемого алгоритма - 5.06ms.

Исходя из кадров, представленных на рис.4.1 и рис.4.2, можно заметить, что предлагаемый эвристический алгоритм позволяет бороться с циклическими колебаниями, связанными с влиянием больших по модулю сил. При этом, исходя из рис.4.6, можно заметить, что использование всех предлагаемых оптимизаций позволяет производить симуляции более чем в два раза быстрее, чем это может быть сделано с использованием библиотеки Nvidia Cloth, признанной стандартом индустрии.

ЗАКЛЮЧЕНИЕ

В рамках проведённого исследования были выполнены все поставленные задачи. Были изучены различные алгоритмы симуляции мягких тел и в частности симуляции тканей. Помимо упомянутого в названии работы эвристического алгоритма, были также разработаны некоторые оптимизации для применения алгоритма XPBD в реализации для графического процессора. Все разработанные алгоритмы были реализованы в рамках модуля для демонстрационного проекта, использующего современные алгоритмы компьютерной графики. Данный модуль также позволяет осуществлять динамическое переключение алгоритма симуляции.

С использованием демонстрационного проекта были проведены эксперименты, в ходе которых были показаны преимущества использования предлагаемых оптимизаций и алгоритмов перед решением из библиотеки Nvidia Cloth, признанной стандартом для симуляции поведения тканей. При этом, при реализации алгоритмов симуляции был сохранен интерфейс библиотеки Nvidia Cloth. Визуальные результаты работы демонстрационного проекта были признаны качественными.

Таким образом, благодаря использованию описанных оптимизаций и сохранению интерфейса библиотеки Nvidia Cloth, представленная реализация может быть использована в любых современных графических системах для ускорения симуляции тканей и улучшения её визуального качества.

В качестве дальнейшей работы наиболее интересными представляются следующие два направления:

Первое - симуляция тканей, задаваемых готовыми трёхмерными объектами. Несмотря на то, что поверхности тканей можно задавать прямоугольной сеткой, наиболее простым (с точки зрения графических художников) способом их задания является именно использование готовых объектов. В связи с этим, стоит разработать дополнительные алгоритмы для определения оптимальных “веревки” и “заплатки” для произвольных полигональных сеток.

Второе - адаптация предлагаемых алгоритмов для симуляции более общего представления мягких тел. Благодаря разработанным алгоритмам удалось увеличить скорость работы симуляции тканей, однако ткани являются не единственным типом мягких объектов. Предлагаемые методы позволяют оптимизировать вычисления для мягких тел, задаваемых ограничениями сохранения расстояния, однако существуют ограничения сохранения объема и площади, позволяющие симулировать другие мягкие тела, такие как органы или желе.

На основании всего вышеперечисленного, предлагаемые алгоритмы позволяют симулировать и визуализировать высокодетализированные поверхности тканей, при этом обеспечивая эффективное использование вычислительных ресурсов GPU.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. *Cook R. L., Torrance K. E.* A reflectance model for computer graphics // ACM Transactions on Graphics (ToG). — 1982. — Т. 1, № 1. — С. 7—24.
2. *Debevec P.* Rendering with natural light // ACM SIGGRAPH 98 Electronic art and animation catalog. — 1998. — С. 166.
3. Detailed rigid body simulation with extended position based dynamics / M. Müller [и др.] // Computer Graphics Forum. Т. 39. — Wiley Online Library. 2020. — С. 101—112.
4. *Gu Y., Yang K., Lu C.* Constraint solving order in position based dynamics // 2017 2nd International Conference on Electrical, Automation and Mechanical Engineering (EAME 2017). — Atlantis Press. 2017. — С. 191—194.
5. *Hillis W. D., Steele Jr G. L.* Data parallel algorithms // Communications of the ACM. — 1986. — Т. 29, № 12. — С. 1170—1183.
6. *Ikeuchi K.* Lambertian reflectance // Encyclopedia of Computer Vision. — 2014. — С. 441—443.
7. *Kajiya J. T.* The rendering equation // Proceedings of the 13th annual conference on Computer graphics and interactive techniques. — 1986. — С. 143—150.
8. *Kim T.-Y., Chentanez N., Müller-Fischer M.* Long range attachments-a method to simulate inextensible clothing in computer games // Proceedings of the ACM SIGGRAPH/Eurographics Symposium on Computer Animation. — 2012. — С. 305—310.
9. *Macklin M., Müller M., Chentanez N.* XPBD: position-based simulation of compliant constrained dynamics // Proceedings of the 9th International Conference on Motion in Games. — 2016. — С. 49—54.
10. Microfacet models for refraction through rough surfaces / B. Walter [и др.] // Proceedings of the 18th Eurographics conference on Rendering Techniques. — 2007. — С. 195—206.
11. *Müller M.* Blazing Fast Neighbor Search with Spatial Hashing. — 2023. — URL: <https://matthias-research.github.io/pages/tenMinutePhysics/11-hashing.pdf> (дата обращения: 01.05.2025).
12. *Müller M., Chentanez N.* Wrinkle Meshes. // Symposium on Computer Animation. Т. 16. — Madrid, Spain. 2010.
13. Optimized spatial hashing for collision detection of deformable objects. / M. Teschner [и др.] // Vmv. Т. 3. — 2003. — С. 47—54.

14. Position based dynamics / M. Müller [и др.] // Journal of Visual Communication and Image Representation. — 2007. — Т. 18, № 2. — С. 109—118.
15. Position-based dynamics simulator of vessel deformations for path planning in robotic endovascular catheterization / Z. Li [и др.] // Medical Engineering & Physics. — 2022. — Т. 110. — С. 103920.
16. *Schlick C.* An inexpensive BRDF model for physically-based rendering // Computer graphics forum. Т. 13. — Wiley Online Library. 1994. — С. 233—246.
17. *Servin M., Lacoursiere C., Melin N.* Interactive simulation of elastic deformable materials // Proceedings of SIGRAD Conference. — Citeseer. 2006. — С. 22—32.
18. Small steps in physics simulation / M. Macklin [и др.] // Proceedings of the 18th annual ACM siggraph/eurographics symposium on computer animation. — 2019. — С. 1—7.
19. *Verlet L.* Computer"experiments"on classical fluids. I. Thermodynamical properties of Lennard-Jones molecules // Physical review. — 1967. — Т. 159, № 1. — С. 98.
20. *Wang Y., Wu X., Wang G.* An angle bending constraint model for position-based dynamics // 2014 International Conference on Virtual Reality and Visualization. — IEEE. 2014. — С. 430—434.
21. *Парусов В. А.* Оптимизация количества вызовов отрисовки в современных графических системах: выпускная квалификационная работа бакалавра: направление 01.03. 02 «Прикладная математика и информатика»; образовательная программа 01.03. 02_02 «Системное программирование». — 2023.
22. *Прасолов В. В.* Задачи и теоремы линейной алгебры. — Наука. Гл. ред. физ.-мат. лит., 1996.